In [1]:
```
import pandas as pd
import numpy as np
from time import sleep
from random import uniform
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import seaborn as sns
import matplotlib.pyplot as plt
from selenium.webdriver.chrome.options import Options
import undetected_chromedriver as uc
from bs4 import BeautifulSoup
import re
import asyncio
import nest_asyncio
import aiohttp
from multiprocessing import Pool
import os
import matplotlib.pyplot as plt
import seaborn as sns
import glob
from collections import Counter
import tensorflow as tf
from tensorflow import keras
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder, MultiLabelBinarize
from sklearn.ensemble import RandomForestRegressor
import keras_tuner as kt
from tensorflow.keras import layers, models
import joblib

import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
```

In [2]:
```
house_types = {
    'Apartment': 'apartment',
    'Flat': 'flat',
    'Condominium': 'condominium',
    'Serviced Residence': 'serviced-residence',
    '1-sty Terrace/Link House': '1-sty-terrace-link-house',
    '2-sty Terrace/Link House': '2-sty-terrace-link-house',
    '3-sty Terrace/Link House': '3-sty-terrace-link-house',
    '4-sty Terrace/Link House': '4-sty-terrace-link-house',
    '1.5-sty Terrace/Link House': '1-5-sty-terrace-link-house',
    '2.5-sty Terrace/Link House': '2-5-sty-terrace-link-house',
    '3.5-sty Terrace/Link House': '3-5-sty-terrace-link-house',
    '4.5-sty Terrace/Link House': '4-5-sty-terrace-link-house',
    'Townhouse': 'townhouse',
    'Cluster House': 'cluster-house',
```

```
    'Bungalow': 'bungalow',
    'Semi-detached House': 'semi-detached-house'
}
```

In [3]:
```
# options = Options()
# options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleW
# driver = uc.Chrome(options=options)

# for house_type, house_type_url in house_types.items():
#     data = []
#     existing_records = set()
#     max_properties = 1500
#     page_number = 0

#     while True:
#         driver.get(f'https://www.iproperty.com.my/sale/{house_type_url}/?page={pa
#         print(f"Scraping {house_type}, Page: {page_number}")

#         try:
#             WebDriverWait(driver, 10).until(
#                 EC.presence_of_element_located((By.XPATH, "//div[contains(@class,
#             )
#         except Exception:
#             print(f"Failed to load data on page {page_number} for {house_type}. E
#             page_number += 1
#             continue

#         houses = driver.find_elements(By.XPATH, "//li[contains(@class, 'ListingsL
#         print(f"Found {len(houses)} houses on page {page_number}")

#         for house in houses:
#             try:
#                 # Skip if it's a false house
#                 try:
#                     false_house = house.find_element(By.XPATH, ".//li[contains(@c
#                     if false_house:
#                         continue
#                 except:
#                     pass

#                 # Title
#                 try:
#                     title = house.find_element(By.XPATH, ".//h2[contains(@class,
#                 except:
#                     continue

#                 # Price
#                 try:
#                     price_text = house.find_element(By.XPATH, ".//li[contains(@cl
#                     price = re.sub(r"[^\d]", "", price_text)
#                 except:
#                     continue

#                 # Price per square foot
#                 try:
#                     price_per_sqft_text = house.find_element(By.XPATH, ".//div[co
```

```
#                    price_per_sqft_match = re.search(r"\d+(\.\d+)?", price_per_sq
#                    price_per_sqft = float(price_per_sqft_match.group()) if price_
#                except:
#                    continue

#                # Size
#                try:
#                    details_text = house.find_element(By.XPATH, ".//div[contains(
#                    built_up_match = re.search(r"Built-up\s*:\s*([\d,]+)\s*sq\. f
#                    size = int(built_up_match.group(1).replace(",", "")) if built_
#                except:
#                    continue

#                # District, State
#                try:
#                    location = house.find_element(By.XPATH, ".//div[contains(@cla
#                    district = location.split(",")[0].title()
#                    state = location.split(",")[1].title()
#                except:
#                    continue

#                # Facilities: Bedrooms, Bathrooms, Car Slots
#                try:
#                    bed_number_text = house.find_element(By.XPATH, ".//li[contain
#                    bed_number = sum(int(num.strip()) for num in bed_number_text.
#                except:
#                    bed_number = None

#                try:
#                    bath_number_text = house.find_element(By.XPATH, ".//li[contai
#                    bath_number = sum(int(num.strip()) for num in bath_number_tex
#                except:
#                    bath_number = None

#                try:
#                    car_number_text = house.find_element(By.XPATH, ".//li[contain
#                    car_number = sum(int(num.strip()) for num in car_number_text.
#                except:
#                    car_number = None

#                try:
#                    link = house.find_elements(By.XPATH, ".//a[@class='depth-list
#                    driver.get(link)
#                    WebDriverWait(driver, 10).until(
#                        EC.presence_of_element_located((By.TAG_NAME, "main"))
#                    )

#                    # Location
#                    try:
#                        location = driver.find_element(By.XPATH, ".//h3[@class='s
#                        location = location.lstrip('-').strip()
#                    except:
#                        location = None

#                    property_types = driver.find_elements(By.XPATH, ".//div[@clas
```

```
#                       # Furnished Type
#                       furnished_type = "Unfurnished"
#                       try:
#                           for property_type in property_types:
#                               key = property_type.find_element(By.XPATH, ".//div[@c
#                               if key == "Furnishing":
#                                   furnished_type = property_type.find_element(By.XP
#                                   break
#                       except:
#                           pass

#                       # Tenure
#                       tenure = "Freehold"
#                       try:
#                           for property_type in property_types:
#                               key = property_type.find_element(By.XPATH, ".//div[@c
#                               if key == "Tenure":
#                                   tenure = property_type.find_element(By.XPATH, "./
#                                   break
#                       except:
#                           pass

#                       # Facilities
#                       try:
#                           facilities = [
#                               facility.get_attribute("textContent").strip().title()
#                               for facility in driver.find_elements(By.XPATH, ".//di
#                           ]
#                       except:
#                           facilities = None

#                       # Images
#                       try:
#                           img_urls = set()
#                           modal_button = WebDriverWait(driver, 10).until(
#                               EC.element_to_be_clickable((By.XPATH, '//div[contains
#                           )
#                           modal_button.click()
#                           WebDriverWait(driver, 10).until(
#                               EC.presence_of_element_located((By.XPATH, '//div[cont
#                           )

#                           images = driver.find_elements(By.XPATH, '//div[contains(@
#                           for img in images:
#                               src = img.get_attribute('src')
#                               if src:
#                                   img_urls.add(src)

#                           close_button = driver.find_element(By.XPATH, '//div[conta
#                           close_button.click()
#                       except Exception as e:
#                           # print(f"Error extracting images: {e}")
#                           img_urls = []

#                   except:
#                       continue
```

```
#                 finally:
#                     driver.back()

#                 record = (
#                     title, price, district, state, bed_number, location, furnishe
#                 )

#                 if record in existing_records:
#                     continue

#                 existing_records.add(record)

#                 data.append({
#                     "Title": title,
#                     "Price": price,
#                     "District": district,
#                     "State": state,
#                     "Bedrooms": bed_number,
#                     "Location": location,
#                     "Tenure": tenure,
#                     "Furnished Type": furnished_type,
#                     "Size": size,
#                     "Facilities": facilities,
#                     "Bathrooms": bath_number,
#                     "Car Slots": car_number,
#                     "House Type": house_type,
#                     "Price per sqft": price_per_sqft,
#                     "Images": img_urls,
#                 })
#             except Exception as e:
#                 print(f"Error extracting house details: {e}")

#         if len(data) >= max_properties:
#             print(f"Reached max properties limit ({max_properties}) for {house_ty
#             break

#         try:
#             next_page_button = driver.find_element(By.XPATH, "//a[@aria-label='Go
#             if next_page_button:
#                 page_number += 1
#                 print(len(data))
#         except Exception:
#             print(f"No 'next page' button found on page {page_number}. Ending scr
#             break

#     output_directory = "data"
#     if not os.path.exists(output_directory):
#         os.makedirs(output_directory)

#     file_name = f"{output_directory}/property-{house_type_url}.csv"
#     df = pd.DataFrame(data[:max_properties])
#     df.to_csv(file_name, index=False, encoding="utf-8")
#     print(f"Data saved to {file_name}")

# driver.quit()
```

In [4]:
```
folder_path = 'data'
all_files = []
for house_type in house_types.values():
    pattern = os.path.join(folder_path, f'property-{house_type}.csv')
    all_files.extend(glob.glob(pattern))

combined_df = pd.concat([pd.read_csv(file) for file in all_files], ignore_index=Tru
combined_file_path = 'data/combined_property_data.csv'
combined_df.to_csv(combined_file_path, index=False)
```

In [5]:
```
df = combined_df
df.head()
```

Out[5]:

	Title	Price	District	State	Bedrooms	Location	Tenure	Furnished Type	S
0	Austin Regency (Pangsapuri Austin Perdana), Ta...	680000	Tebrau	Johor	4.0	Jalan Austin Perdana Utama, Taman Austin Perda...	Freehold	Fully Furnished	131
1	Pangsapuri Seri Baiduri, Perling	218000	Perling	Johor	3.0	Jalan Baiduri 3, 80100, Johor	Freehold	Unfurnished	75
2	Sri Intan 1, Jalan Kuching	258000	Jalan Kuching	Kuala Lumpur	3.0	No.2 Jalan Trolak 6, 51200, Kuala Lumpur	Freehold	Partly Furnished	90
3	Mentari Court, Petaling Jaya	290000	Petaling Jaya	Selangor	3.0	Jalan PJS8/9, 46150, Selangor	Leasehold	Partly Furnished	77
4	Bistari Impian Apartment, Taman Dato Onn, Joho...	350000	Johor Bahru	Johor	3.0	0 Jalan Serantau, Taman Dato Onn, 80350, Johor	Leasehold	Unfurnished	107

In [6]:
```
len(df)
```

Out[6]: 19462

In [7]:
```
df = df.drop_duplicates(subset=df.columns.difference(['Facilities']))
```

In [9]:
```
print(len(df))
```

```
19462
```

In [10]:
```
# Identify missing values in critical columns
critical_columns = ['Price', 'Size', 'Bedrooms', 'Bathrooms', 'Car Slots', 'Distric
missing_summary_critical = df[critical_columns].isnull().sum()
missing_summary_critical
```

Out[10]:
```
Price                0
Size               113
Bedrooms           153
Bathrooms           56
Car Slots         6319
District             0
State                0
Price per sqft       8
Furnished Type       0
House Type           0
Facilities           0
dtype: int64
```

In [11]:
```
# Drop rows with missing critical values
df = df.dropna(subset=['Price', 'Size', 'Bedrooms', 'Bathrooms', 'Price per sqft'],

# Drop Unnecessary Columns
columns_to_drop = ['Location', 'Title', 'Images', 'District']
df = df.drop(columns=columns_to_drop, errors='ignore')
```

In [12]:
```
# Impute missing numerical values
numerical_critical = ['Bedrooms', 'Bathrooms', 'Car Slots']
df[numerical_critical] = df[numerical_critical].fillna(df[numerical_critical].media
```

In [13]:
```
# Address Outliers for Numerical Data
numerical_columns = ['Price', 'Size', 'Price per sqft']
for col in numerical_columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
```

In [16]:
```
# Fix Inconsistent Formatting
categorical_columns = ['State', 'Tenure', 'Furnished Type', 'House Type']
df[categorical_columns] = df[categorical_columns].apply(lambda x: x.str.strip().str
```

In [17]:
```
# Create Derived Features
df['Price per Bedroom'] = df['Price'] / df['Bedrooms']
df['Room Density'] = df['Bedrooms'] / df['Size']
```

In [18]:
```
# Convert Categorical Data to Numeric Codes
```

```
for col in df.select_dtypes(include=['object']).columns:
    if col != 'Facilities':
        df[f"{col}_code"] = df[col].astype('category').cat.codes
```

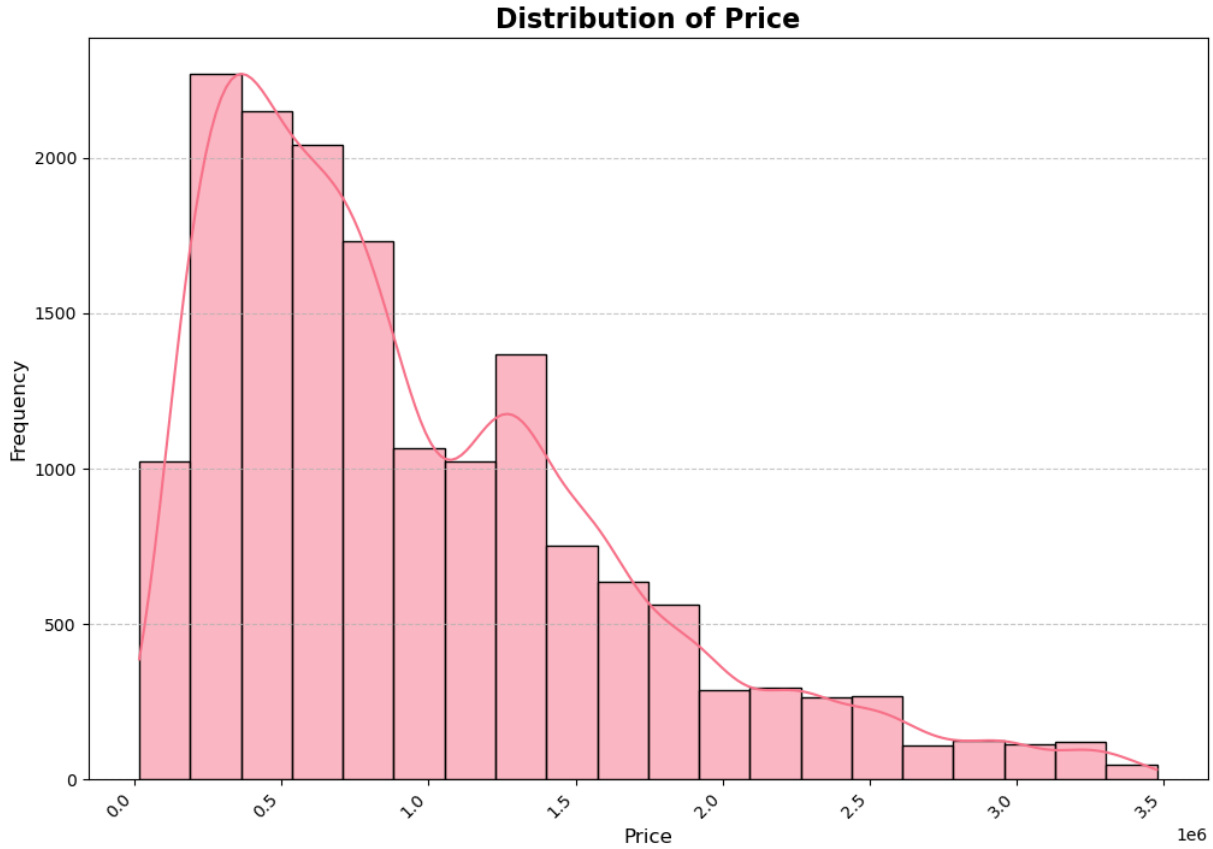
In [19]:
```
# Generate and interpret summary statistics and Validate Data Quality
summary_statistics = df.describe()
summary_statistics
```

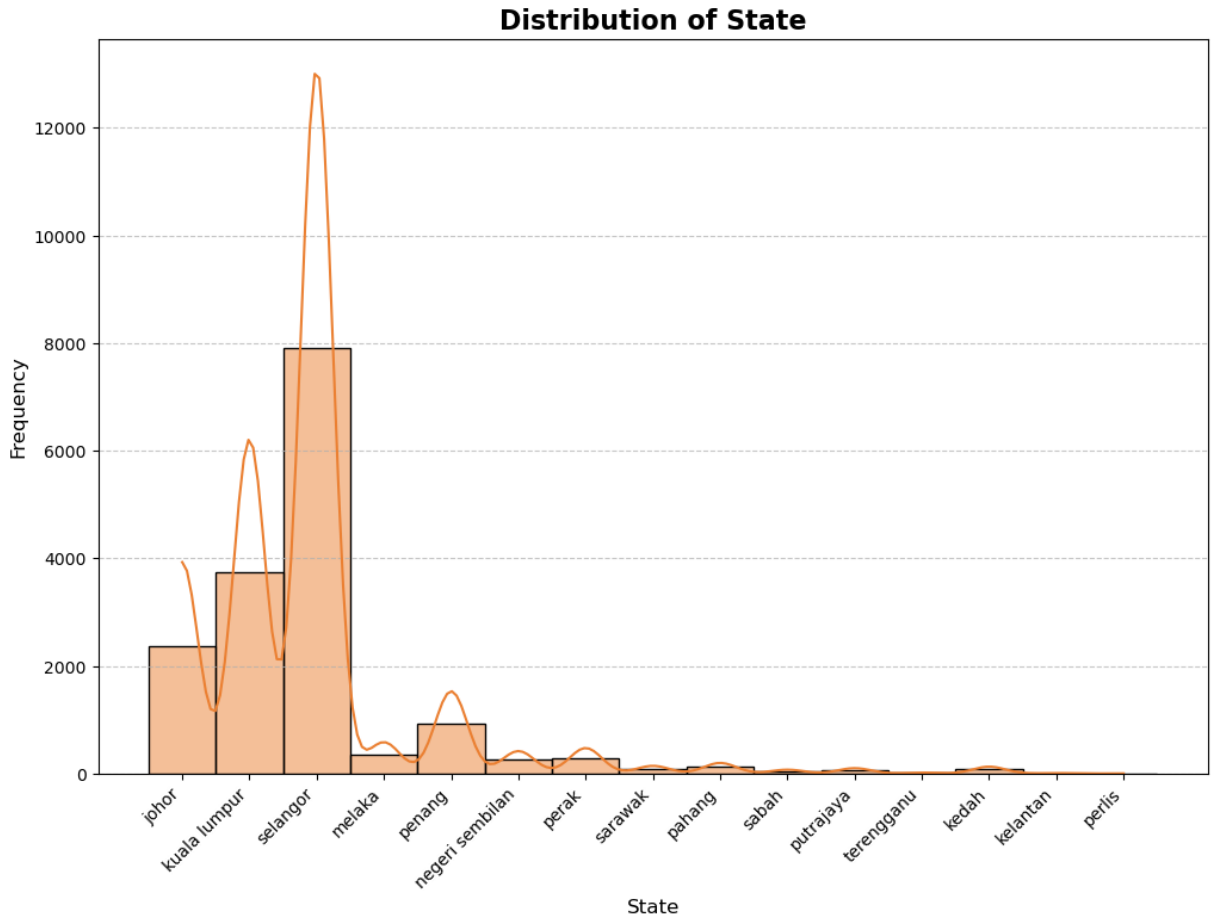
Out[19]:

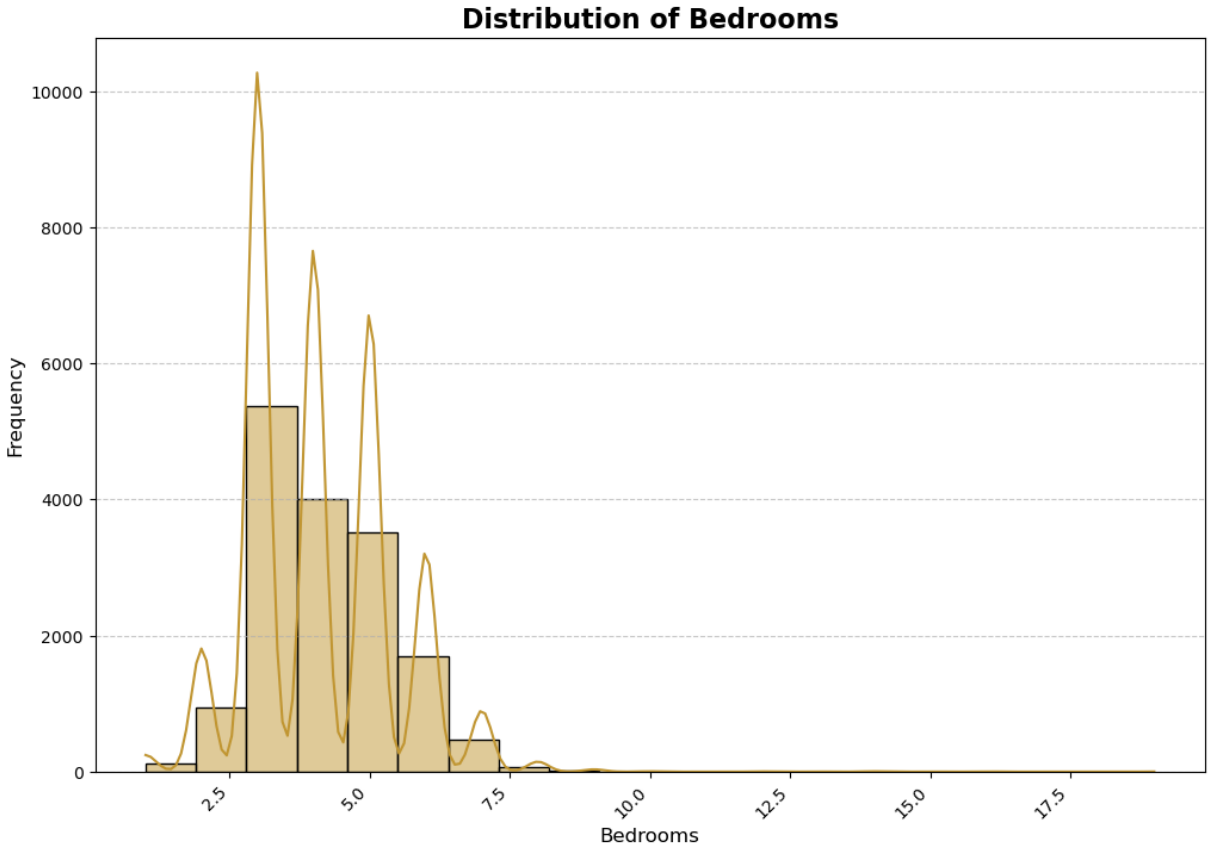
	Price	Bedrooms	Size	Bathrooms	Car Slots	Price per sqft
count	1.625100e+04	16251.000000	16251.000000	16251.000000	16251.000000	16251.000000
mean	9.627161e+05	4.072672	2090.864439	3.255184	2.193219	442.764500
std	6.936631e+05	1.304835	1142.383706	1.465852	0.976628	173.587615
min	1.548800e+04	1.000000	140.000000	1.000000	1.000000	4.160000
25%	4.200000e+05	3.000000	1100.000000	2.000000	2.000000	314.875000
50%	7.680000e+05	4.000000	1900.000000	3.000000	2.000000	414.140000
75%	1.350000e+06	5.000000	2900.000000	4.000000	2.000000	542.045000
max	3.478000e+06	19.000000	5523.000000	15.000000	12.000000	965.000000

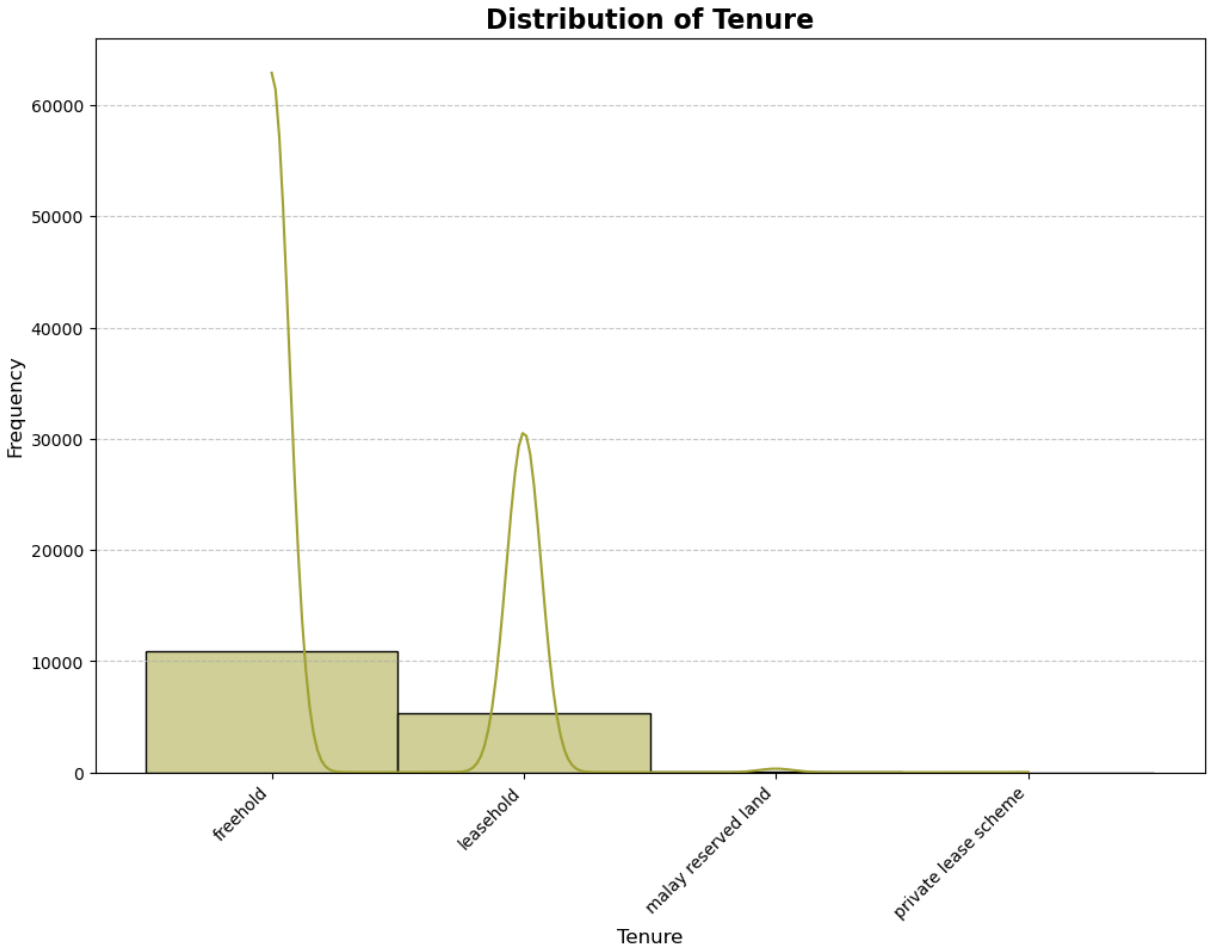
In [20]:
```
# Analyze data distributions for key variables
columns_to_plot = [
    col for col in df.columns
    if col not in ['Facilities', 'State_code', 'Tenure_code', 'Furnished Type_code'
]

colors = sns.color_palette("husl", n_colors=len(columns_to_plot) + 1)

for idx, column in enumerate(columns_to_plot):
    plt.figure(figsize=(12, 8))
    sns.histplot(data=df, x=column, bins=20, kde=True, color=colors[idx], edgecolor
    plt.title(f'Distribution of {column}', fontsize=16, fontweight='bold')
    plt.xlabel(column, fontsize=12)
    plt.ylabel('Frequency', fontsize=12)
    plt.xticks(rotation=45, ha='right')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()

df['Facilities_list'] = df['Facilities'].apply(lambda x: eval(x) if isinstance(x, s
all_facilities = [facility for sublist in df['Facilities_list'] for facility in sub
facility_expanded_df = pd.DataFrame({'Facility': all_facilities})


plt.figure(figsize=(12, 8))
sns.histplot(data=facility_expanded_df, x='Facility', kde=True, bins=20, color=colo
plt.title('Histogram of Facilities Across Houses', fontsize=16, fontweight='bold')
plt.xlabel('Facilities', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.xticks(rotation=45, ha='right')
```

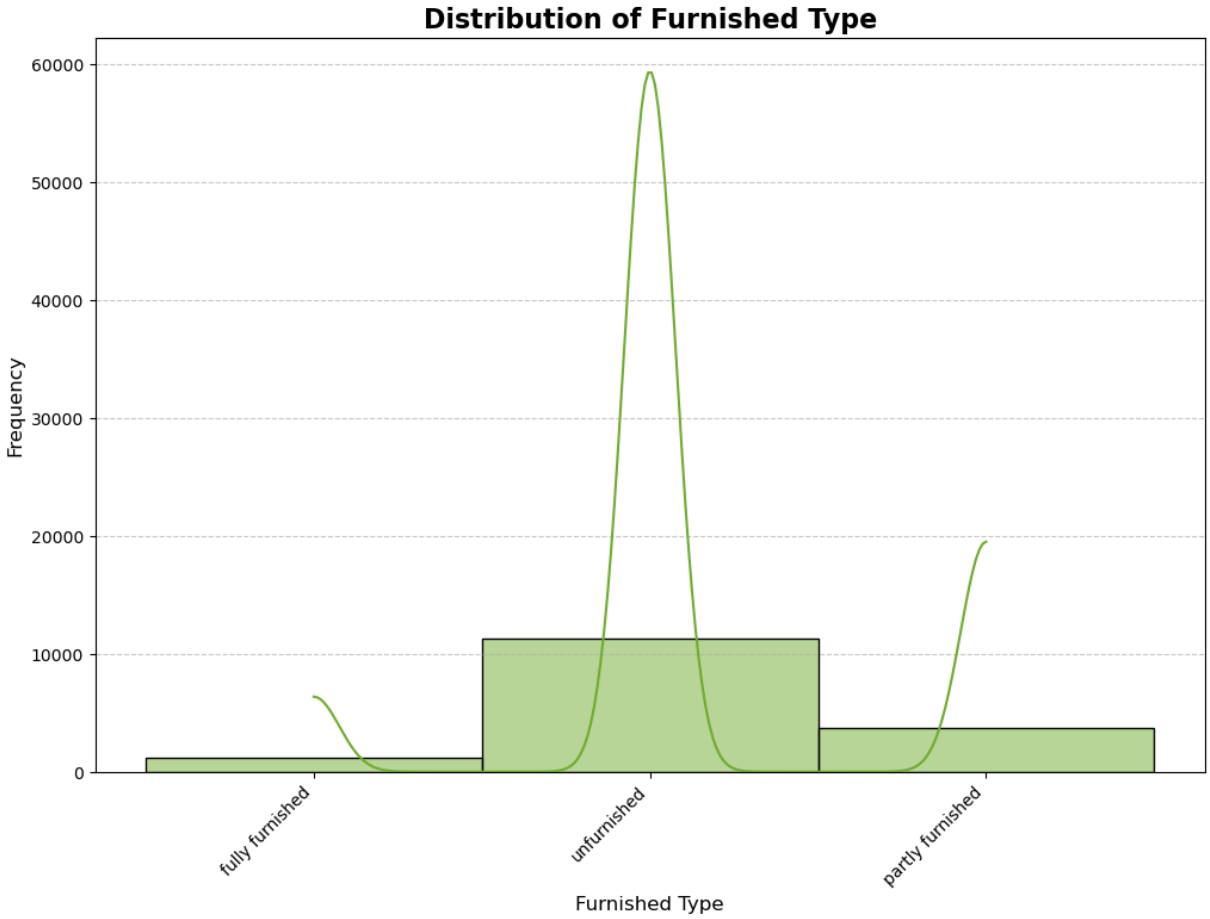
```
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

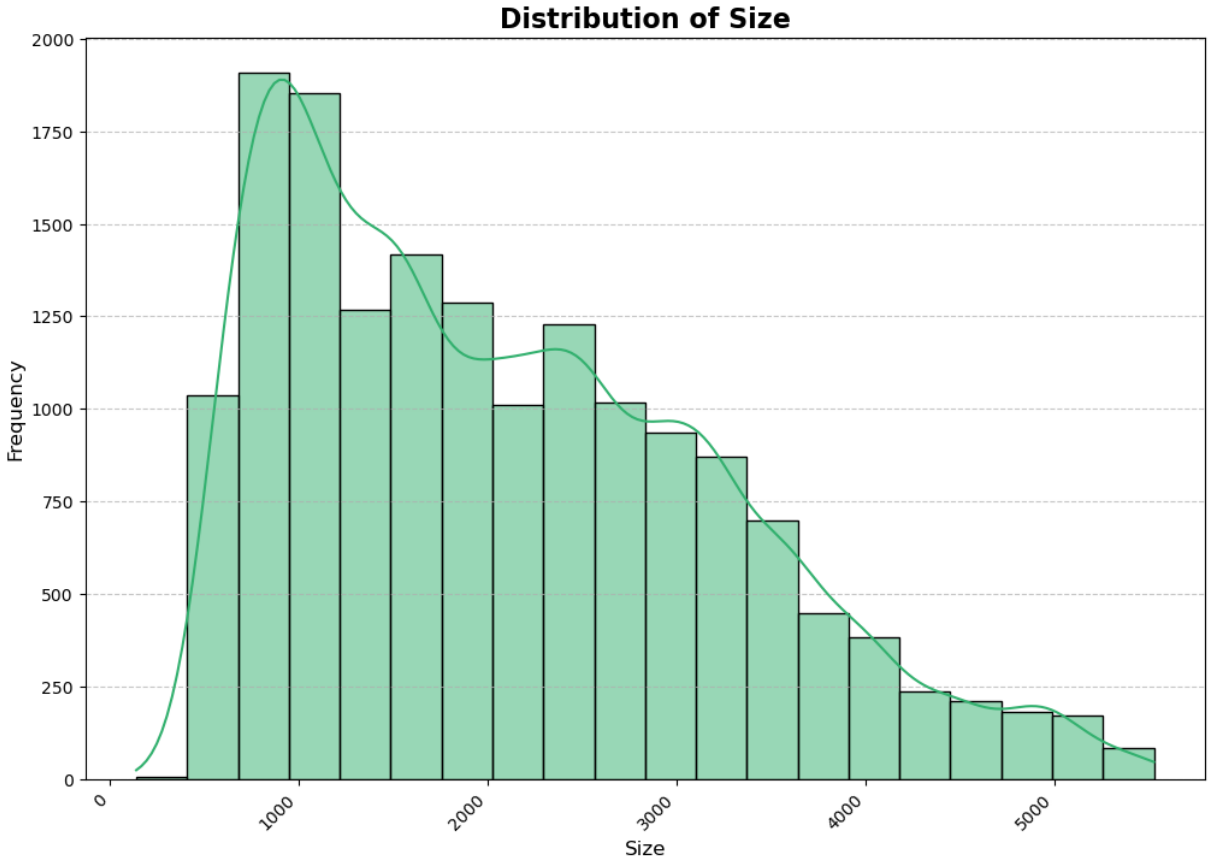


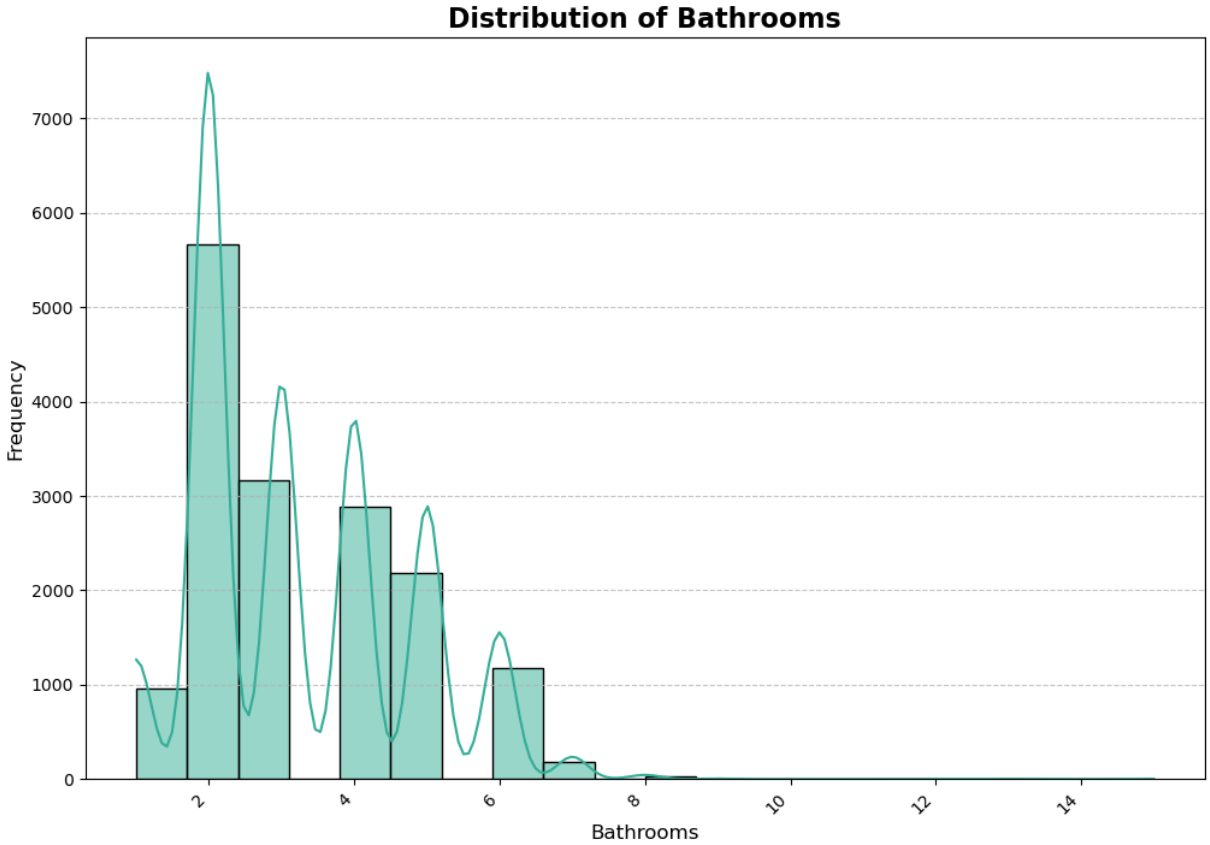


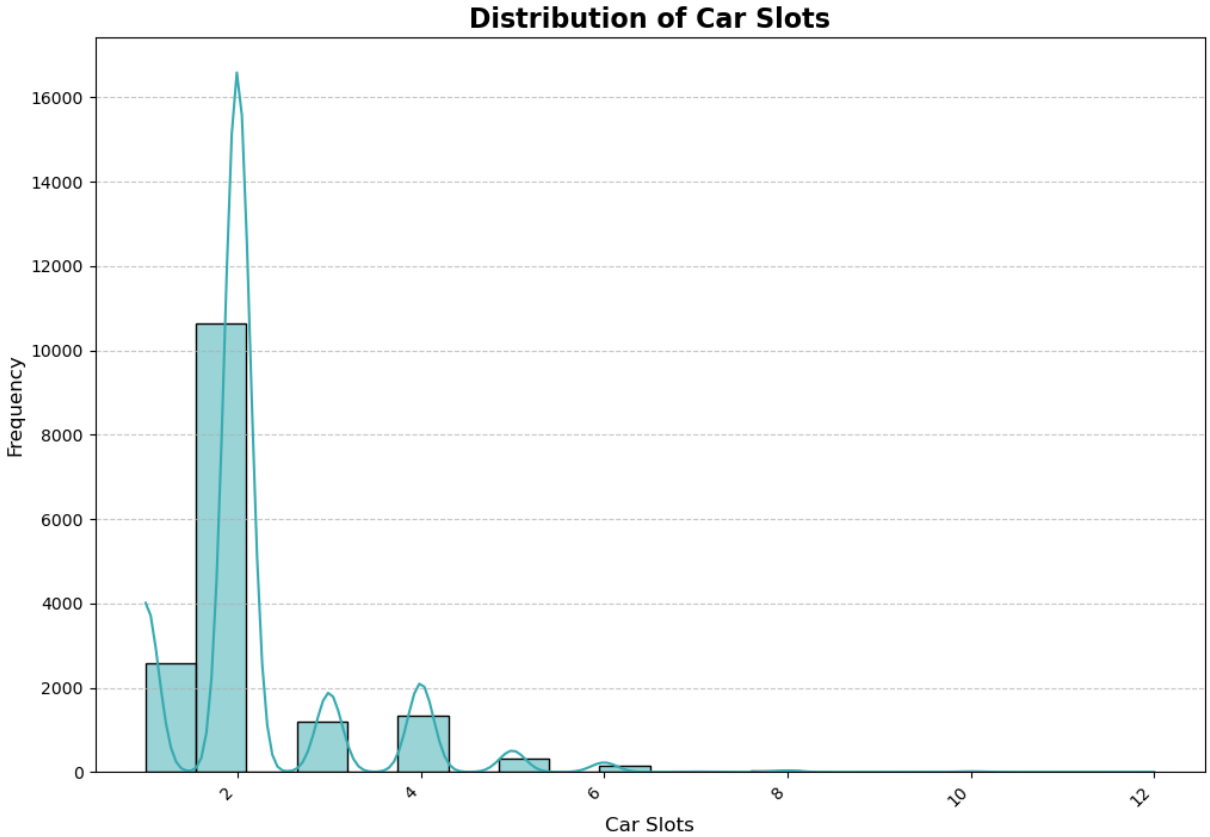

Distribution of Bedrooms
Frequency
10000
8000
6000
4000
2000
0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Bedrooms

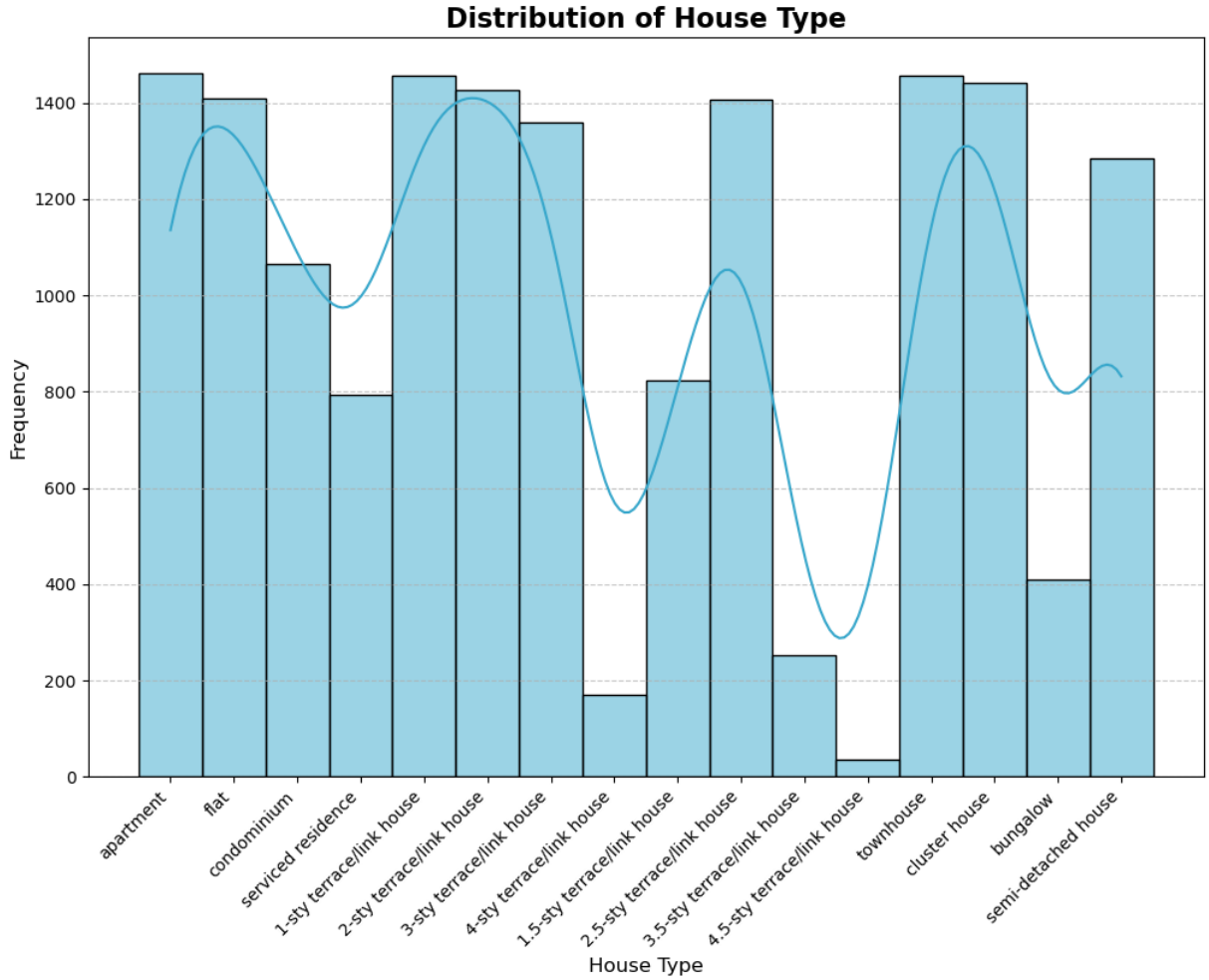

Distribution of Tenure
Frequency
60000
50000
40000
30000
20000
10000
0
freehold
leasehold
malay reserved land
private lease scheme
Tenure

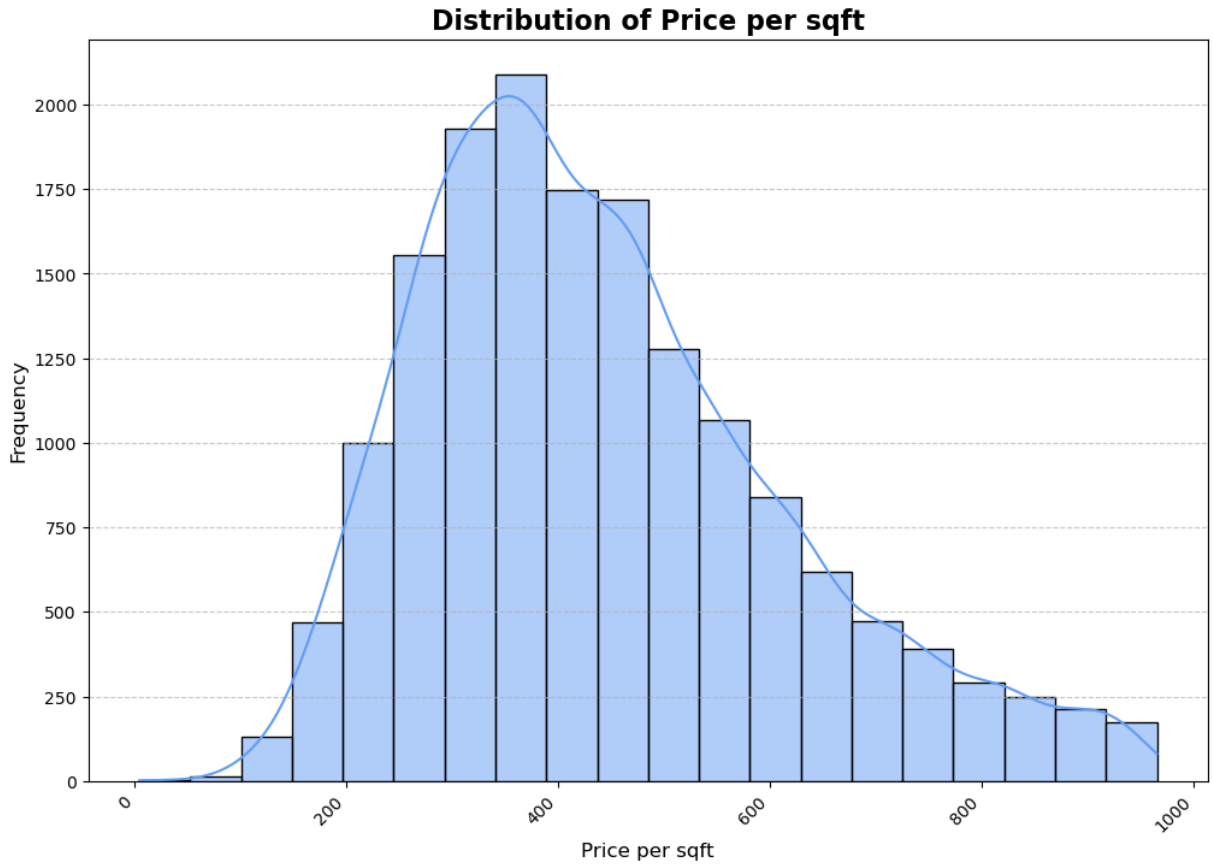

Distribution of Furnished Type
Frequency
60000
50000
40000
30000
20000
10000
0
fully furnished
unfurnished
partly furnished
Furnished Type

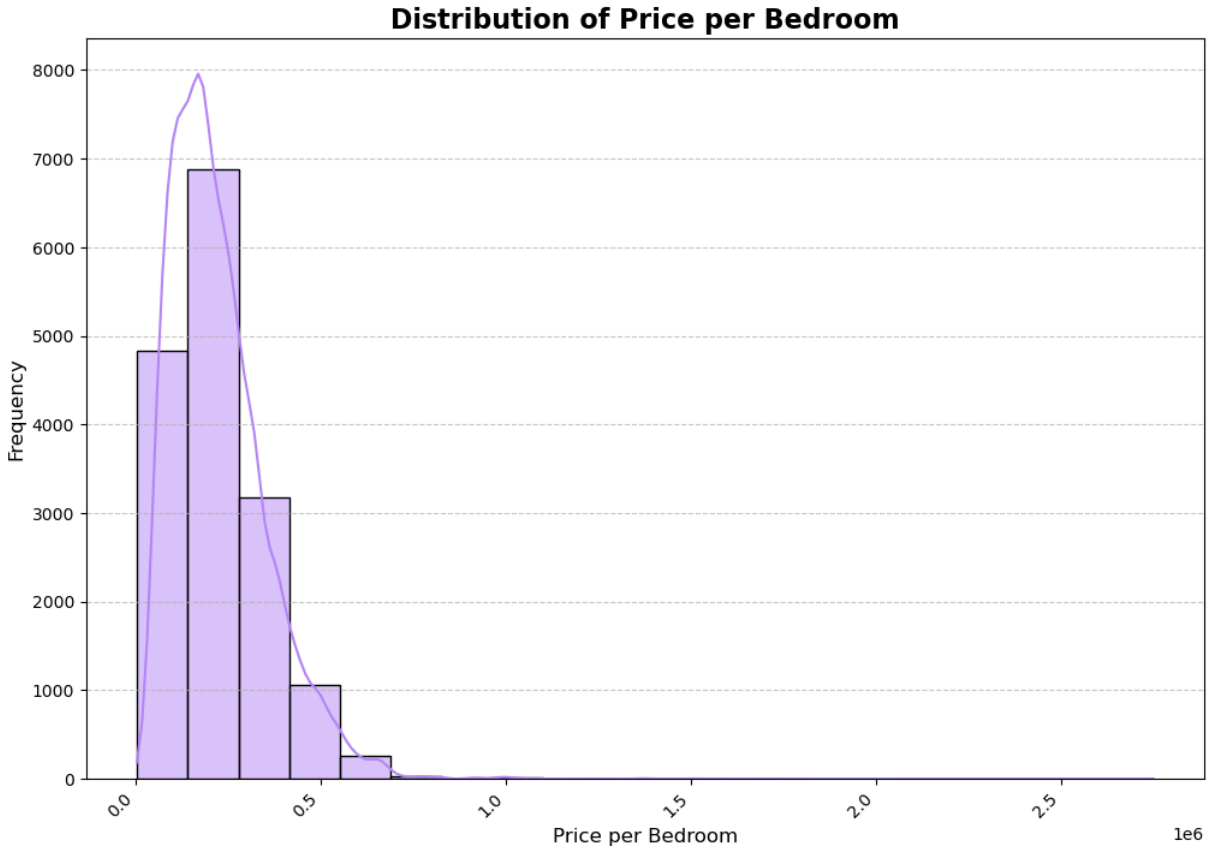

Distribution of Size
Frequency
2000
1750
1500
1250
1000
750
500
250
0
0
1000
2000
3000
4000
5000
Size

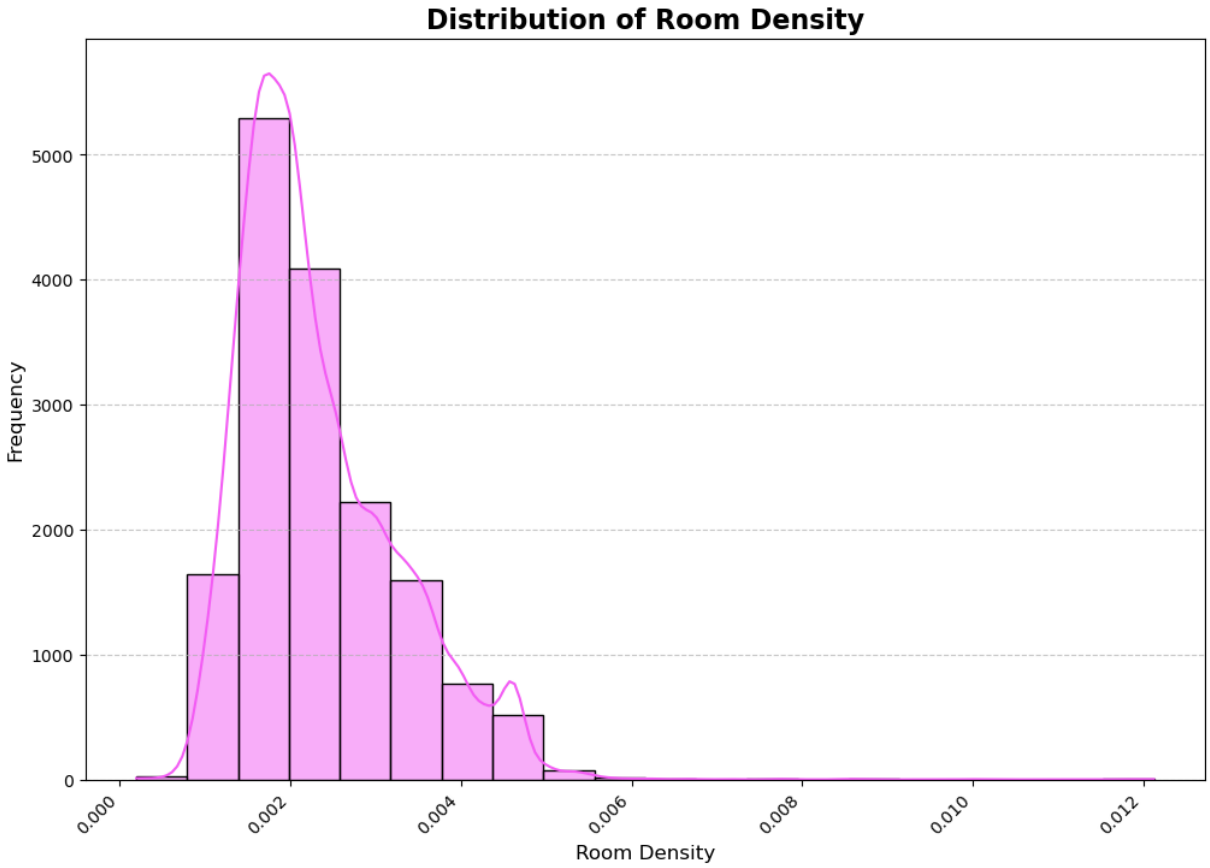

Distribution of Bathrooms
Frequency
0
1000
2000
3000
4000
5000
6000
7000
2
4
6
8
10
12
14
Bathrooms

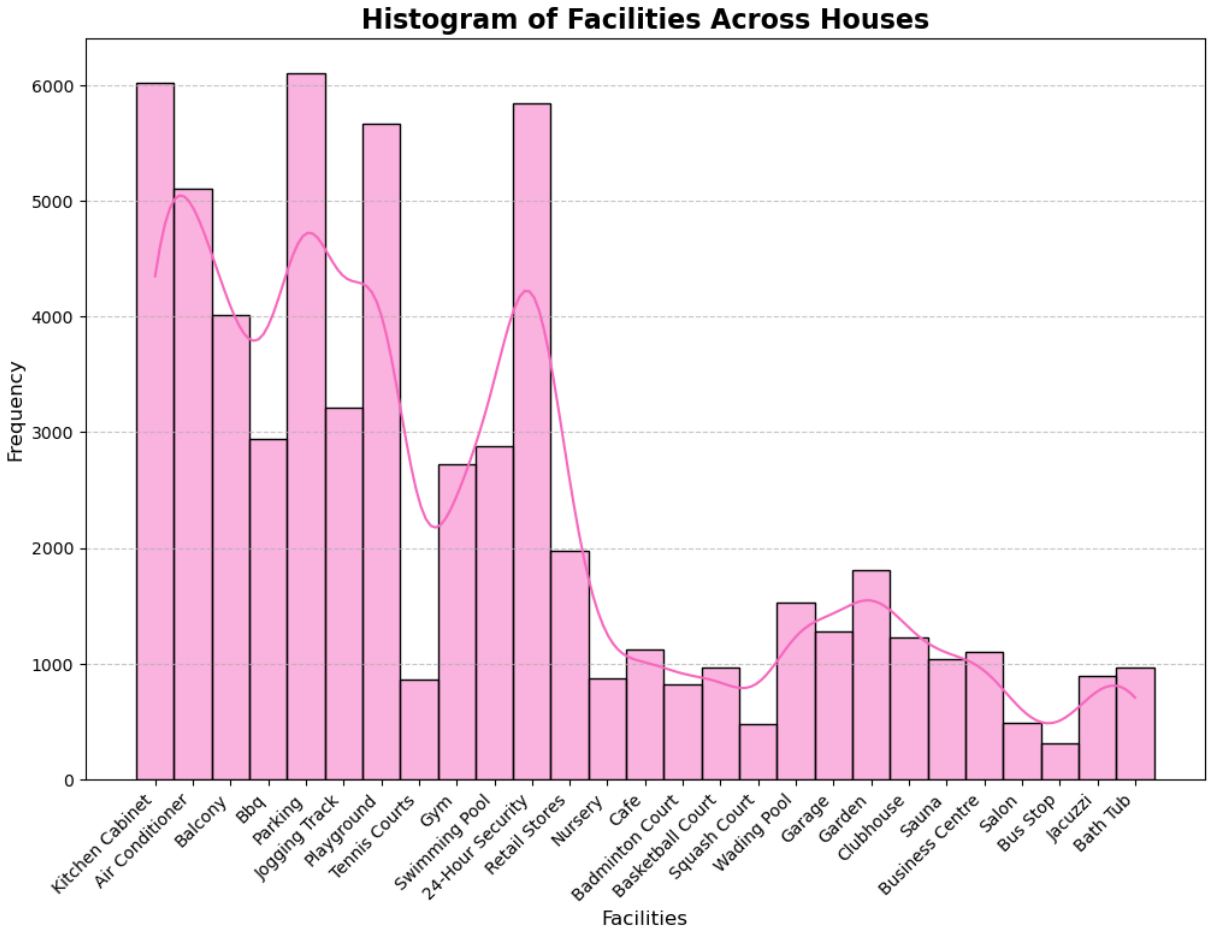

Distribution of Car Slots
Frequency
0
2000
4000
6000
8000
10000
12000
14000
16000
2
4
6
8
10
12
Car Slots


Distribution of House Type
Frequency
0
200
400
600
800
1000
1200
1400
apartment
flat
condominium
serviced residence
1-sty terrace/link house
2-sty terrace/link house
3-sty terrace/link house
4-sty terrace/link house
1.5-sty terrace/link house
2.5-sty terrace/link house
3.5-sty terrace/link house
4.5-sty terrace/link house
townhouse
cluster house
bungalow
semi-detached house
House Type


Distribution of Price per sqft
Frequency
0
250
500
750
1000
1250
1500
1750
2000
0
200
400
600
800
1000
Price per sqft

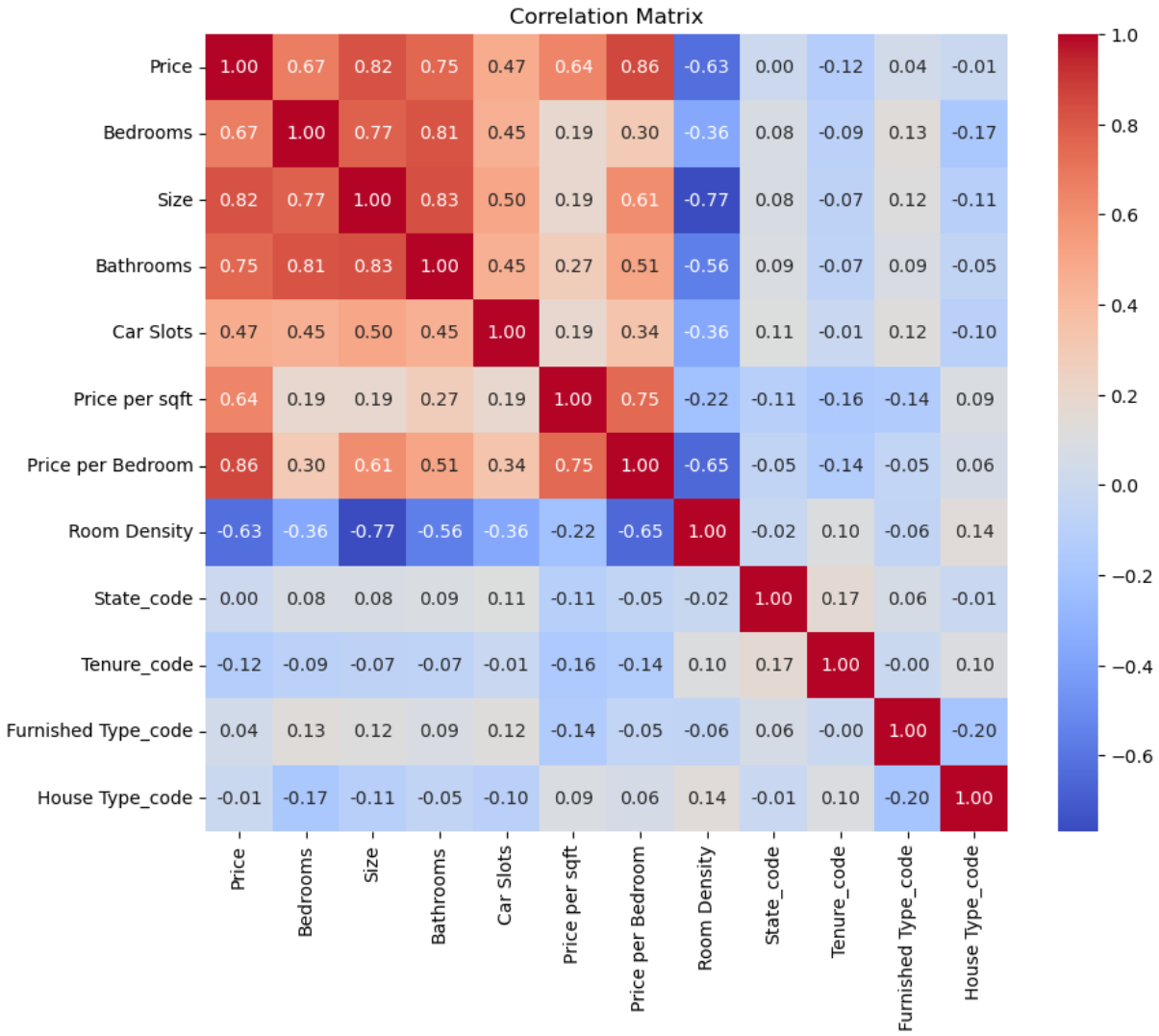

Distribution of Price per Bedroom
Frequency
8000
7000
6000
5000
4000
3000
2000
1000
0
0.0
0.5
1.0
1.5
2.0
2.5
Price per Bedroom
1e6


Distribution of Room Density
Frequency
5000
4000
3000
2000
1000
0
0.000
0.002
0.004
0.006
0.008
0.010
0.012
Room Density



In [21]:
```
# Create correlation analysis
numerical_df = df.select_dtypes(include=['number']).drop(columns=['Facilities_code'
correlation_matrix = numerical_df.corr()
fig3, ax3 = plt.subplots(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm', ax=ax3)
ax3.set_title('Correlation Matrix')
```

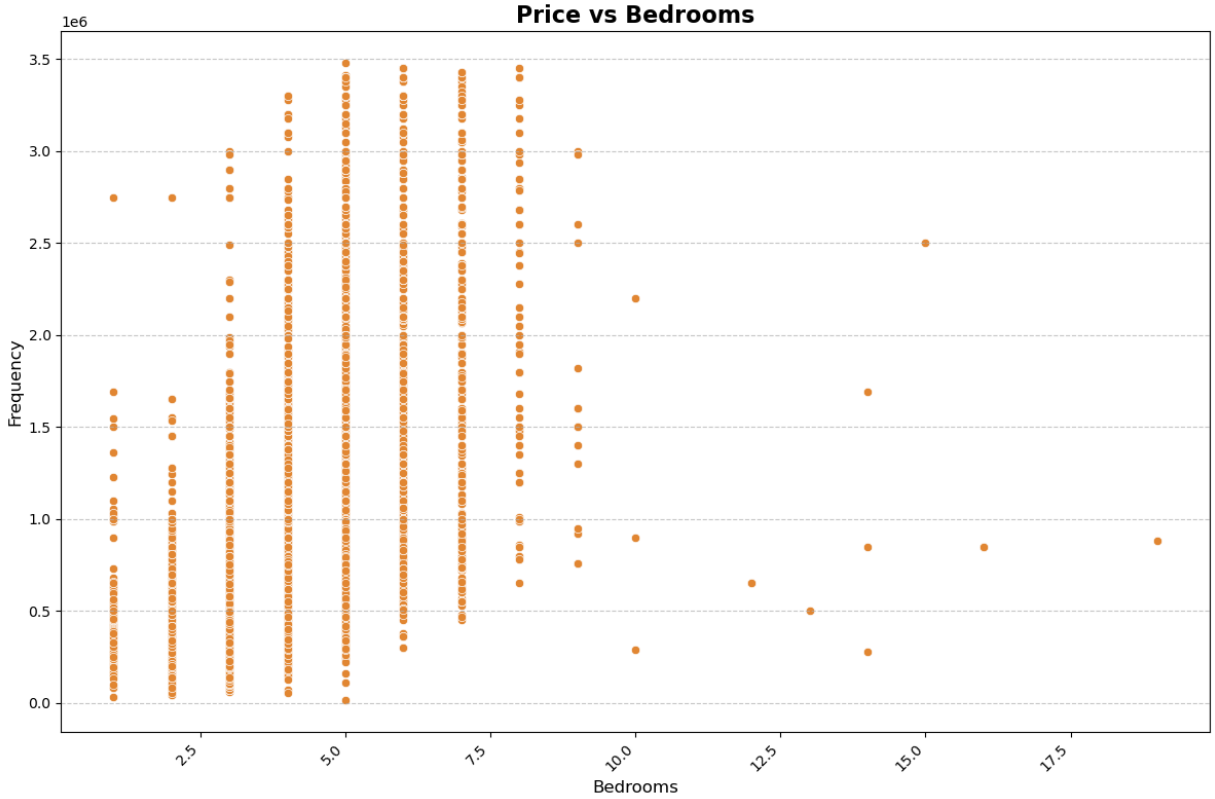
Out[21]:
```
Text(0.5, 1.0, 'Correlation Matrix')
```

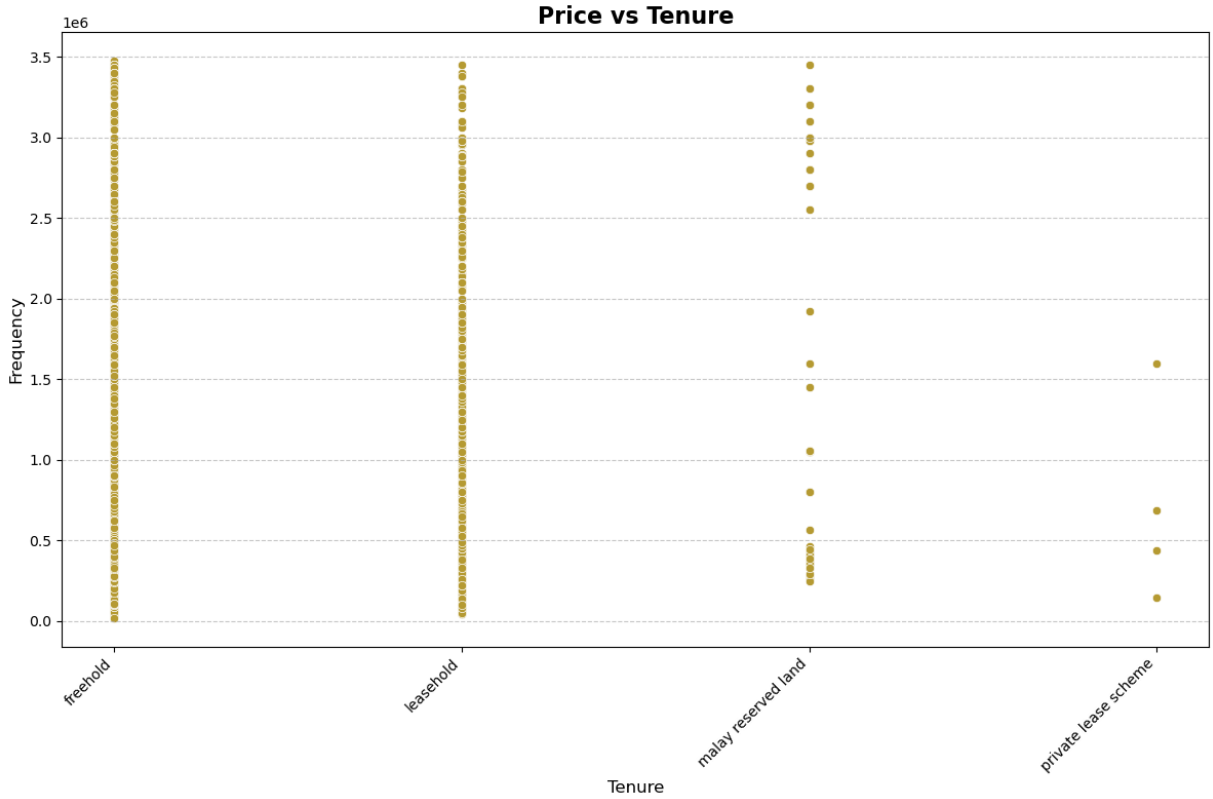


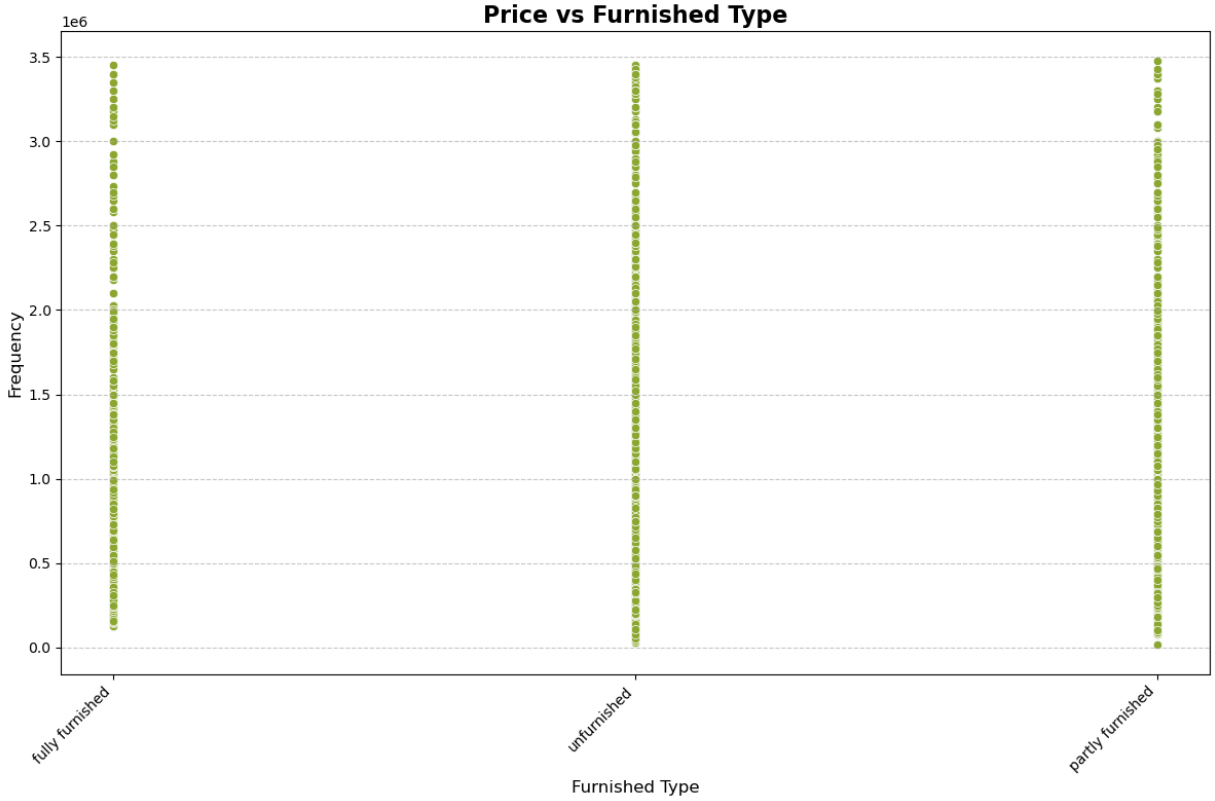
In [22]:

```
# Identify and visualize key trends
columns_to_plot = [
    col for col in df.columns
    if col not in ['Facilities', 'Price', 'State_code', 'Tenure_code', 'Furnished T
]
colors = sns.color_palette("husl", n_colors=len(columns_to_plot))

for idx, column in enumerate(columns_to_plot):
    plt.figure(figsize=(12, 8))
    sns.scatterplot(data=df, x=column, y='Price', color=colors[idx])
    plt.title(f'Price vs {column}', fontsize=16, fontweight='bold')
    plt.xlabel(column, fontsize=12)
    plt.ylabel('Frequency', fontsize=12)
    plt.xticks(rotation=45, ha='right')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.show()
```

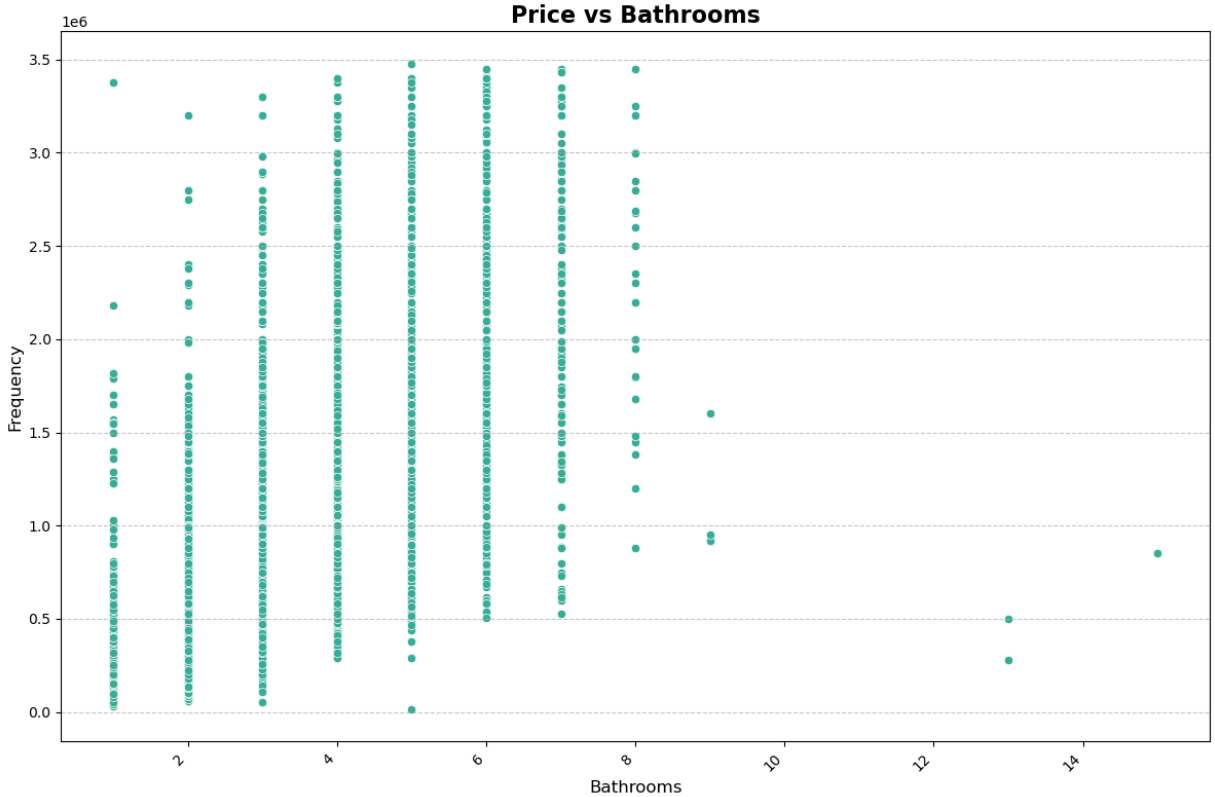
Price vs State
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Frequency
johor
kuala lumpur
selangor
melaka
penang
negeri sembilan
perak
sarawak
pahang
sabah
putrajaya
terengganu
kedah
kelantan
perlis
State

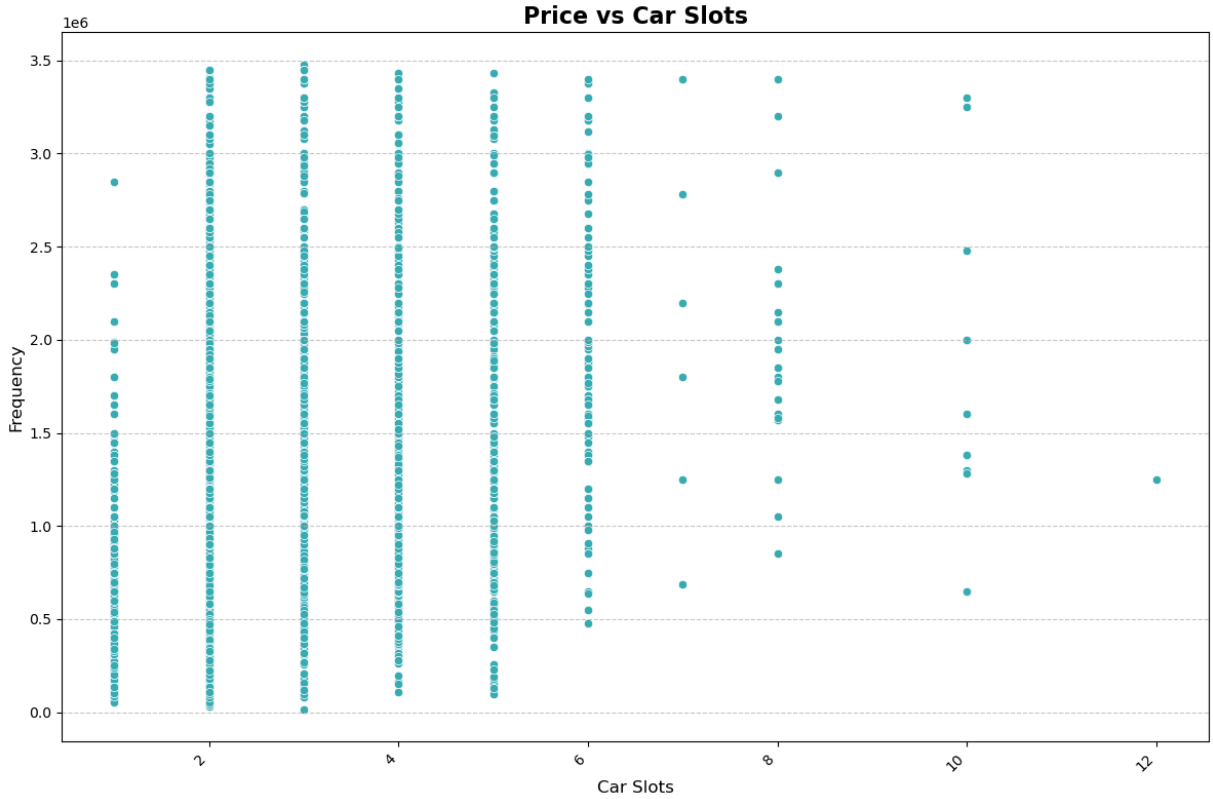

Price vs Bedrooms
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Frequency
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Bedrooms


Price vs Tenure
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Frequency
freehold
leasehold
malay reserved land
private lease scheme
Tenure


Price vs Furnished Type
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Frequency
fully furnished
unfurnished
partly furnished
Furnished Type

Price vs Size
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Frequency
0
1000
2000
3000
4000
5000
Size

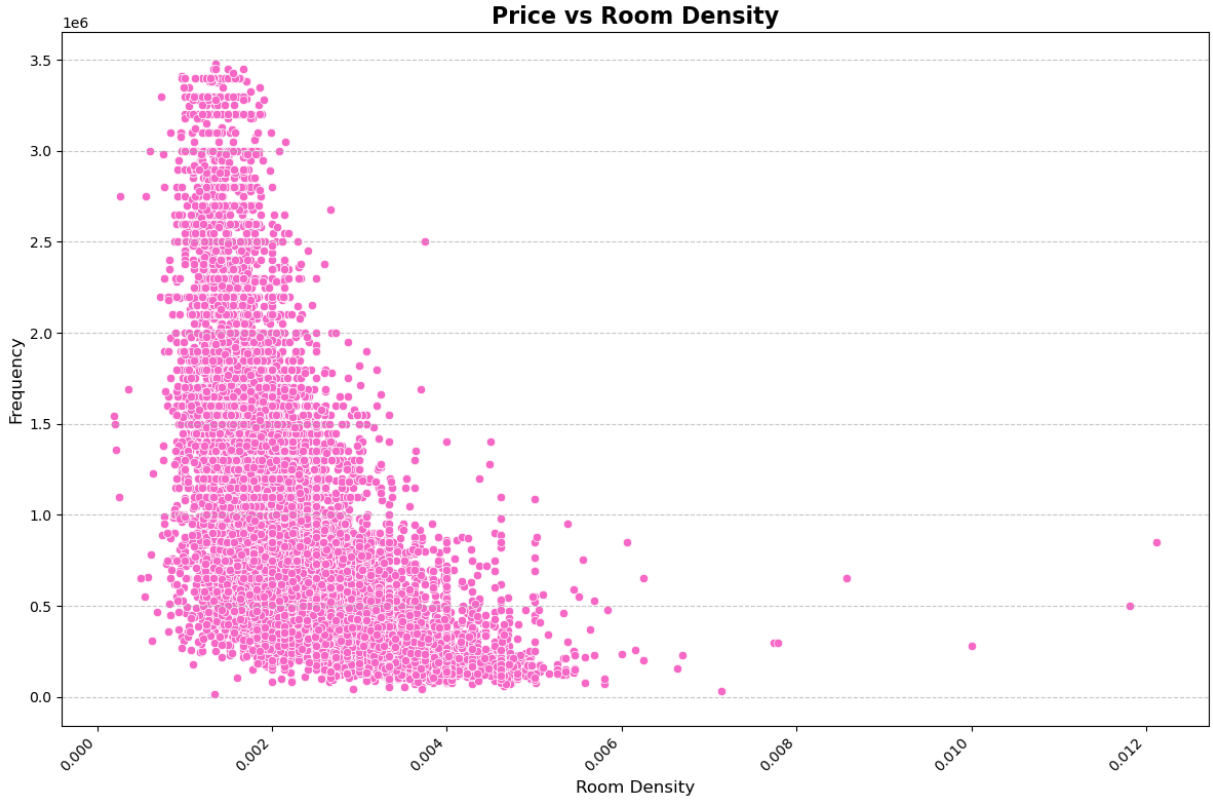

Price vs Bathrooms
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Frequency
2
4
6
8
10
12
14
Bathrooms


Price vs Car Slots
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Frequency
2
4
6
8
10
12
Car Slots

Price vs House Type
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Frequency
apartment
flat
condominium
serviced residence
1-sty terrace/link house
2-sty terrace/link house
3-sty terrace/link house
4-sty terrace/link house
1.5-sty terrace/link house
2.5-sty terrace/link house
3.5-sty terrace/link house
4.5-sty terrace/link house
townhouse
cluster house
bungalow
semi-detached house
House Type

Price vs Price per sqft
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Frequency
0
200
400
600
800
1000
Price per sqft

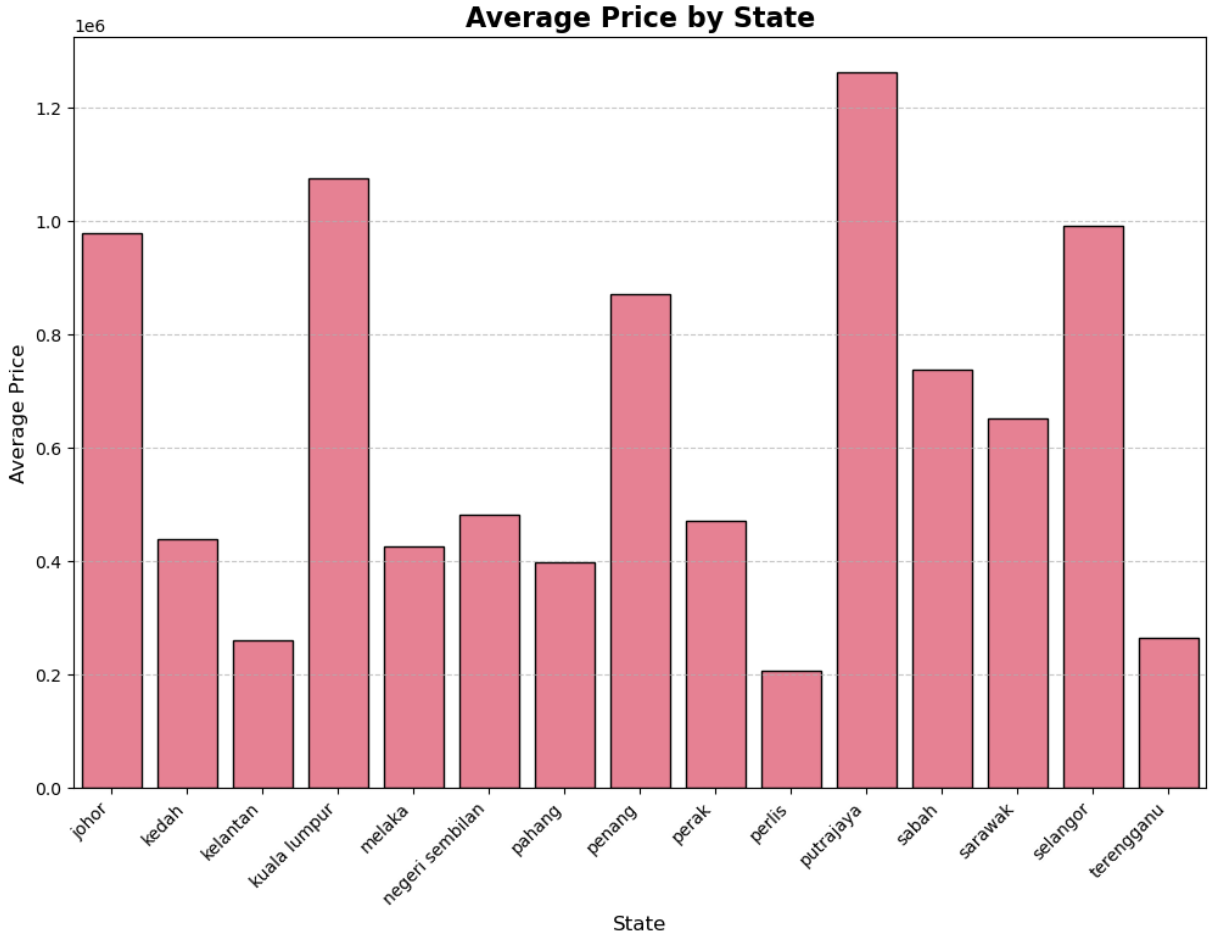
Price vs Price per Bedroom
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Frequency
0.0
0.5
1.0
1.5
2.0
2.5
Price per Bedroom
1e6

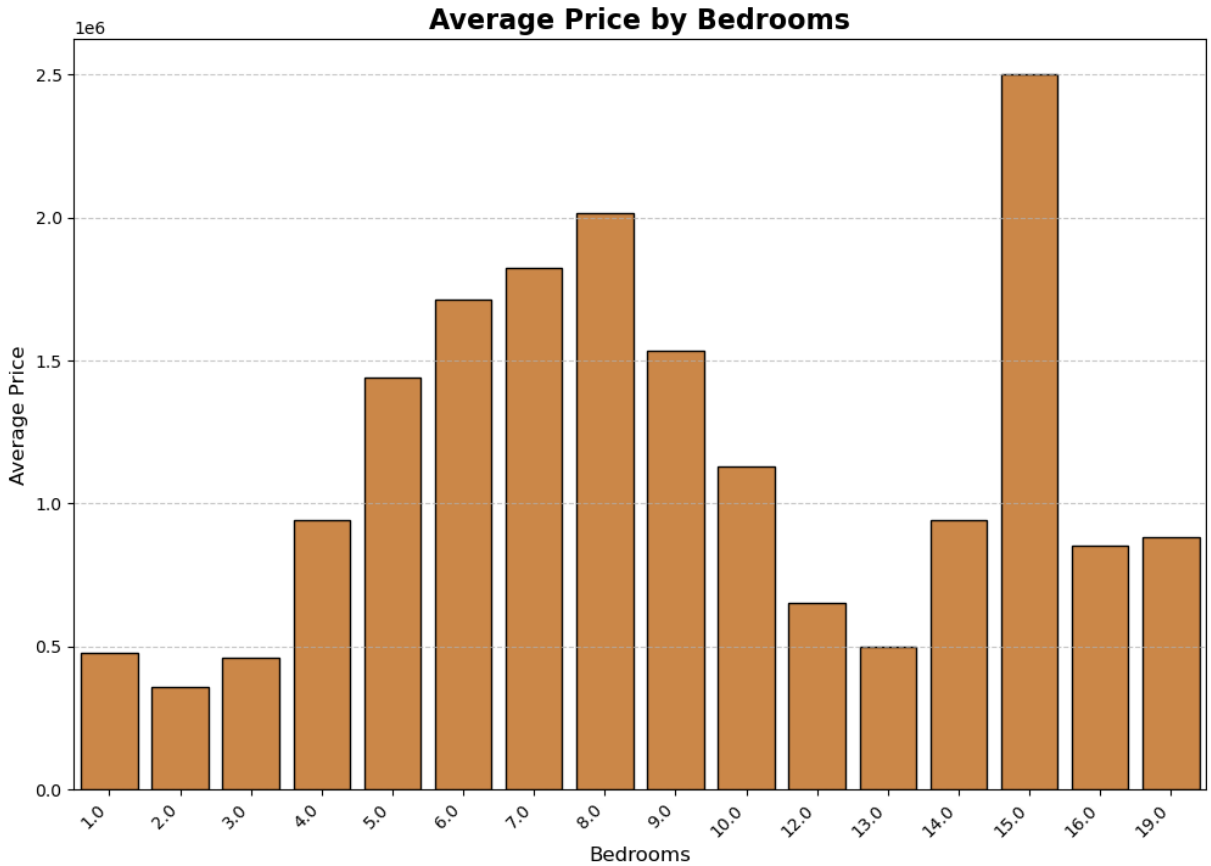


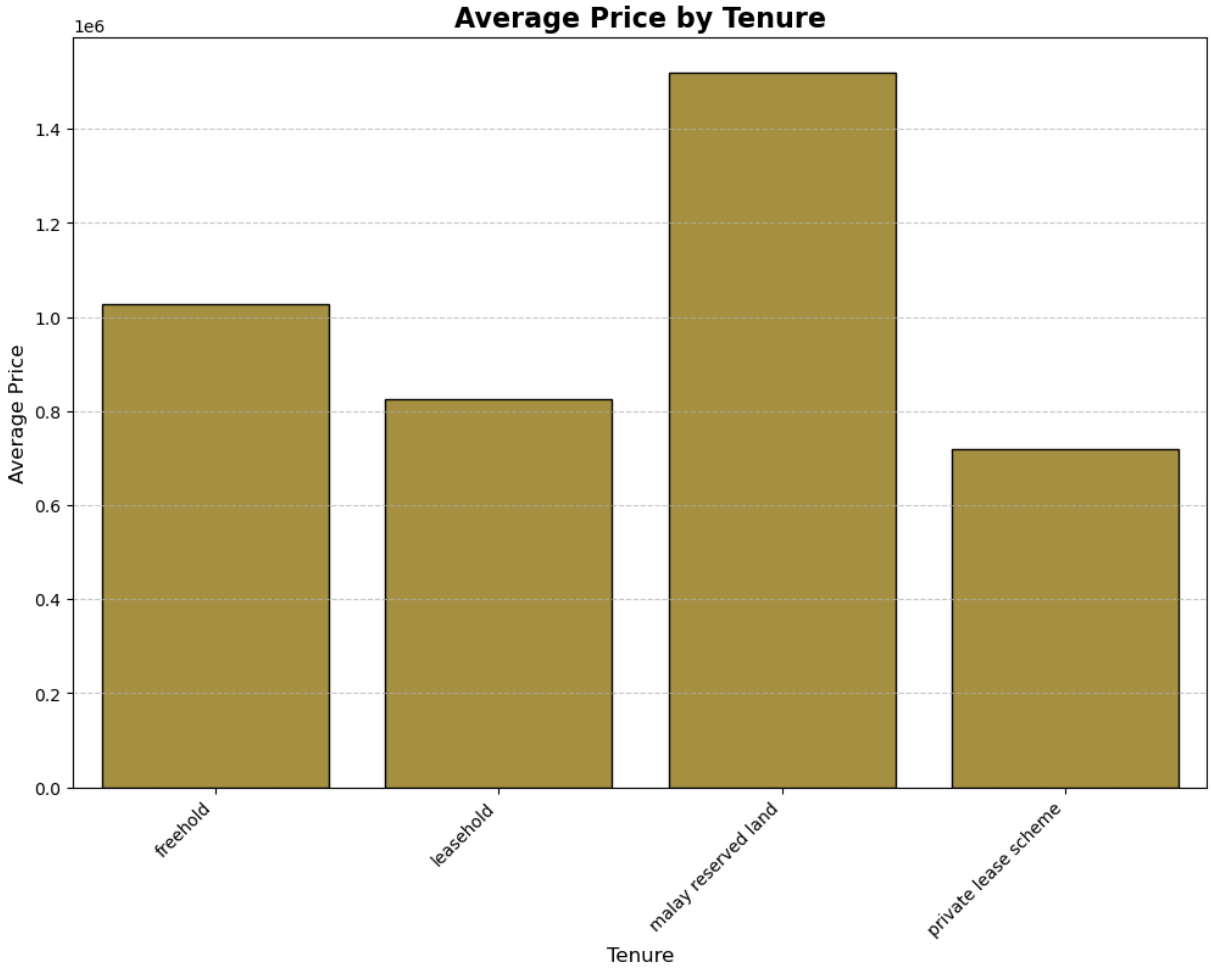
In [23]:
```
size_bins = [0, 500, 1000, 1500, 2000, 2500, df['Size'].max()]
size_labels = ['0-500', '501-1000', '1001-1500', '1501-2000', '2001-2500', '2500+']
df['Size_Bins'] = pd.cut(df['Size'], bins=size_bins, labels=size_labels, include_lo

max_price_per_sqft = df['Price per sqft'].max()
rounded_max_price_per_sqft = np.ceil(max_price_per_sqft / 100) * 100
bins_sqft = np.linspace(0, rounded_max_price_per_sqft, 6)
labels_sqft = [f"{int(bins_sqft[i])}-{int(bins_sqft[i + 1])}" for i in range(len(bi

df['Price_per_Sqft_Bins'] = pd.cut(df['Price per sqft'], bins=bins_sqft, labels=lab

max_price_per_bedroom = df['Price per Bedroom'].max()
rounded_max_price_per_bedroom = np.ceil(max_price_per_bedroom / 100000) * 100000
bins_bedroom = np.linspace(0, rounded_max_price_per_bedroom, 6)
labels_bedroom = [f"{int(bins_bedroom[i])}-{int(bins_bedroom[i + 1])}" for i in ran

df['Price_per_Bedroom_Bins'] = pd.cut(df['Price per Bedroom'], bins=bins_bedroom, l

binned_columns = ['Size_Bins', 'Price_per_Sqft_Bins', 'Price_per_Bedroom_Bins']
binned_labels = ['Size', 'Price per Sqft', 'Price per Bedroom']
binned_colors = colors[-3:]
```

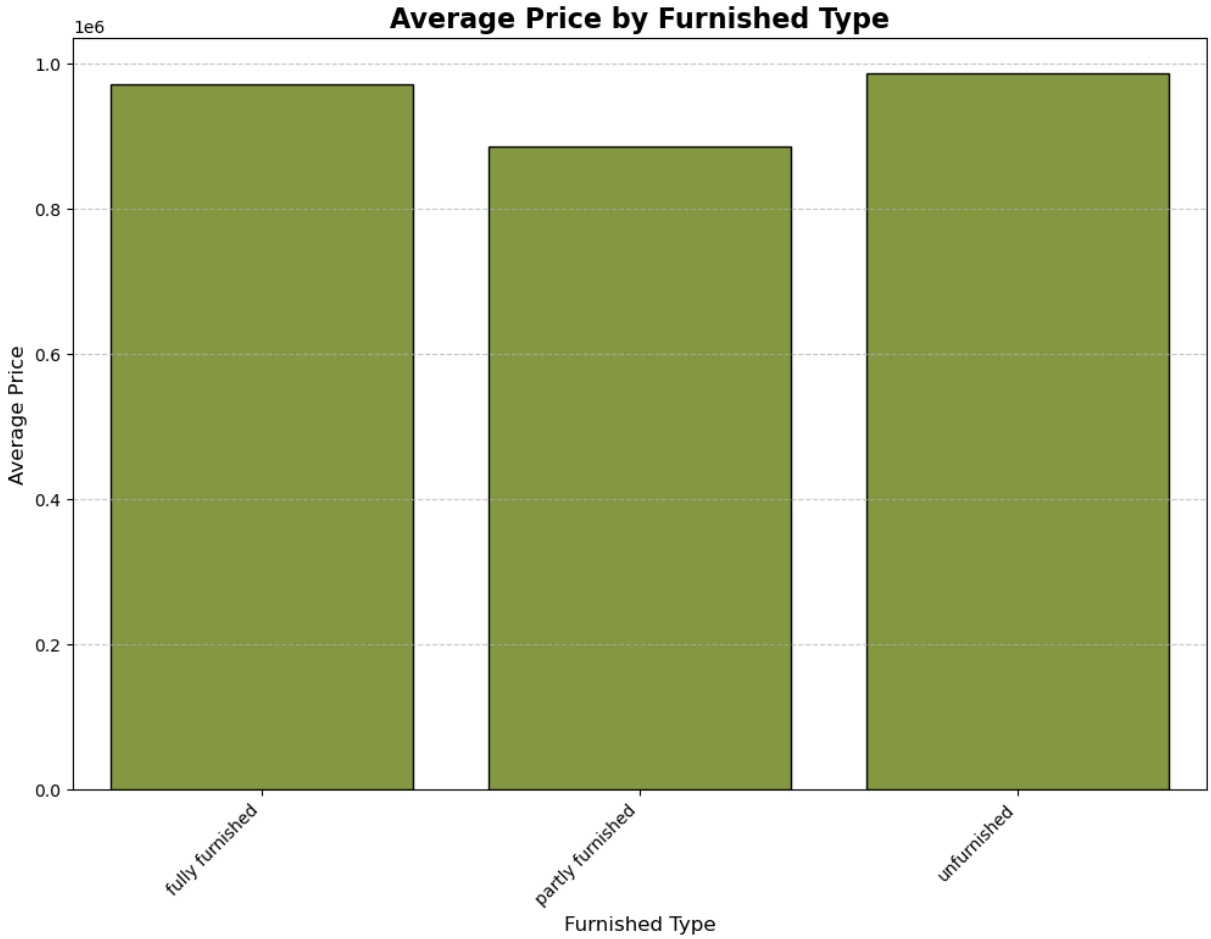
In [24]:
```
# Bar plotting
columns_to_plot = [
    col for col in df.columns
    if col not in ['Facilities', 'Price', 'State_code', 'Tenure_code', 'Furnished T
]
colors = sns.color_palette("husl", n_colors=len(columns_to_plot)+4)

for idx, column in enumerate(columns_to_plot):
    avg_price = df.groupby(column)['Price'].mean().reset_index()
```

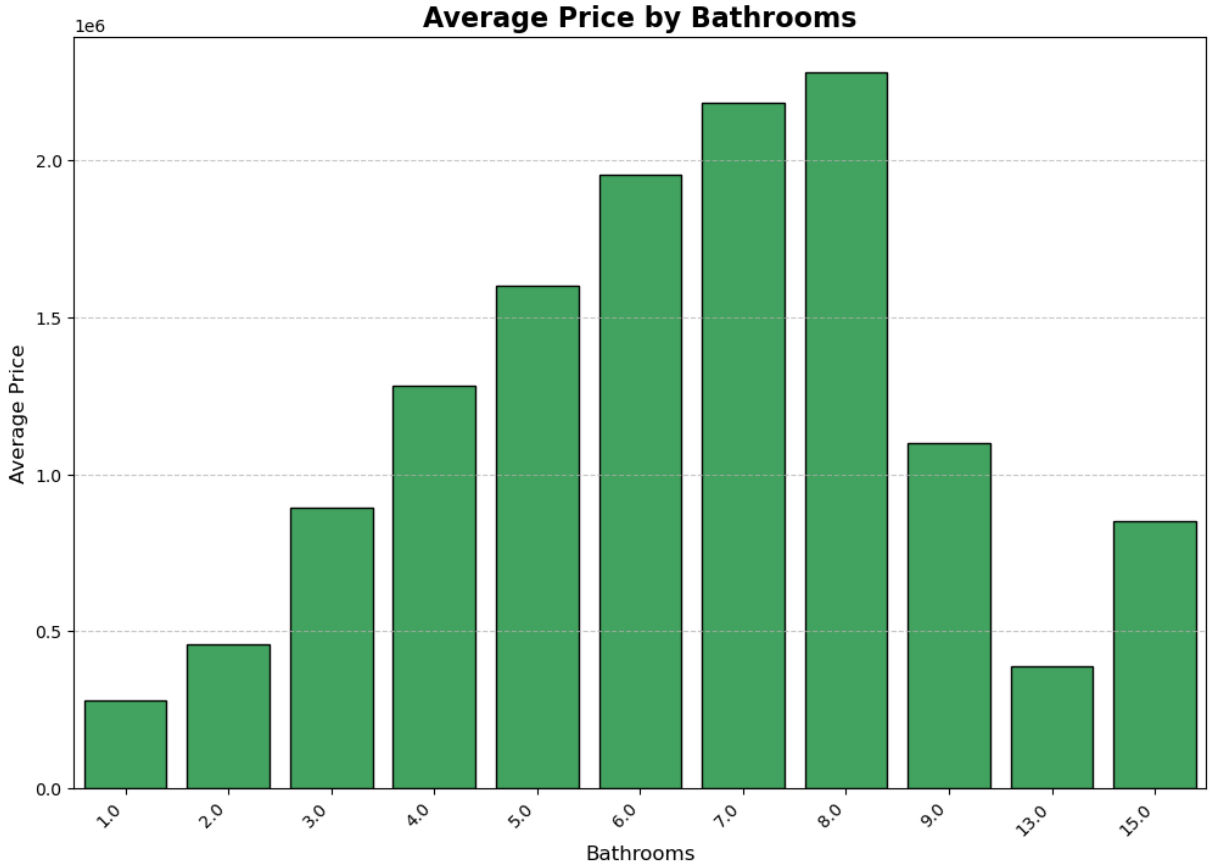
```
    plt.figure(figsize=(12, 8))
    sns.barplot(data=avg_price, x=column, y='Price', color=colors[idx], edgecolor="
    plt.title(f'Average Price by {column}', fontsize=16, fontweight='bold')
    plt.xlabel(column, fontsize=12)
    plt.ylabel('Average Price', fontsize=12)
    plt.xticks(rotation=45, ha='right')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()

for idx, binned_column in enumerate(binned_columns):
    avg_price = df.groupby(binned_column, observed=True)['Price'].mean().reset_inde
    plt.figure(figsize=(12, 8))
    sns.barplot(data=avg_price, x=binned_column, y='Price', color=binned_colors[idx
    plt.title(f'Average Price by {binned_labels[idx]}', fontsize=16, fontweight='bo
    plt.xlabel(binned_labels[idx], fontsize=12)
    plt.ylabel('Average Price', fontsize=12)
    plt.xticks(rotation=45, ha='right')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()
```

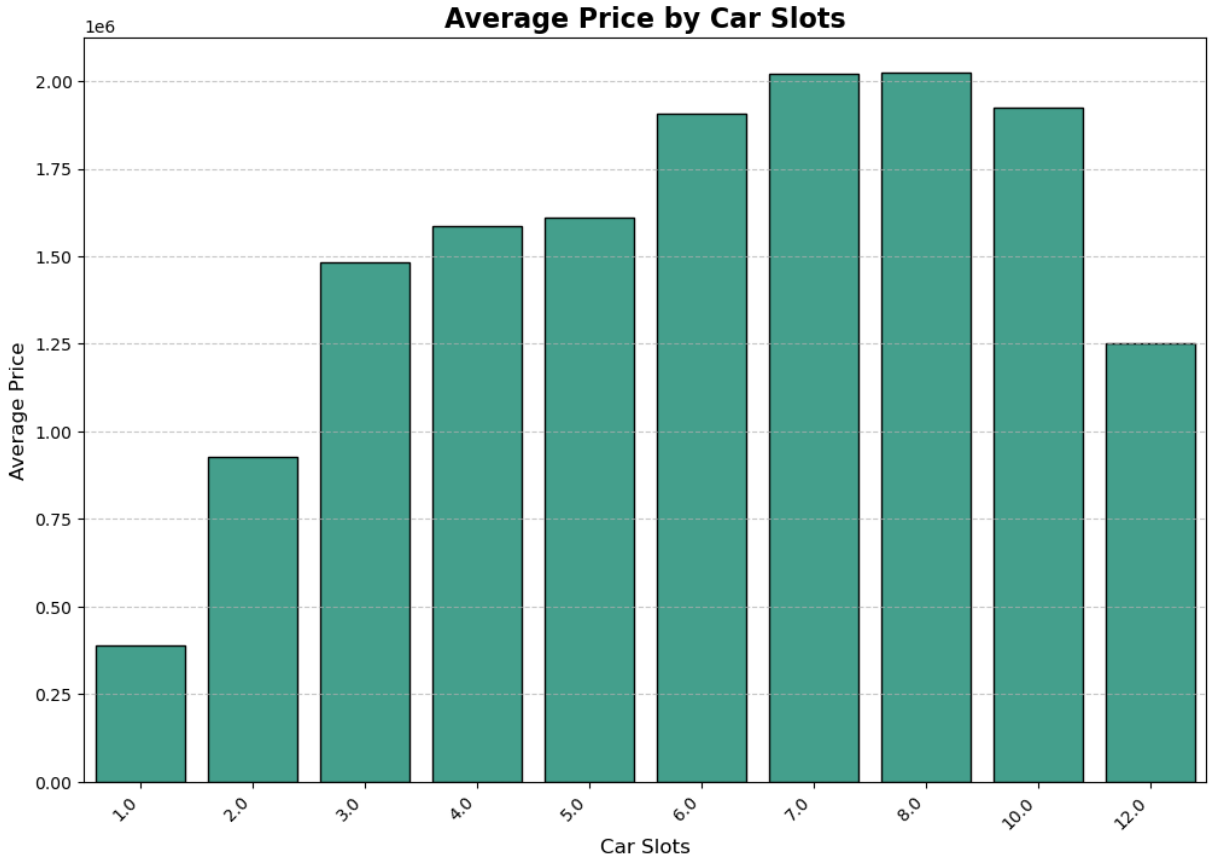


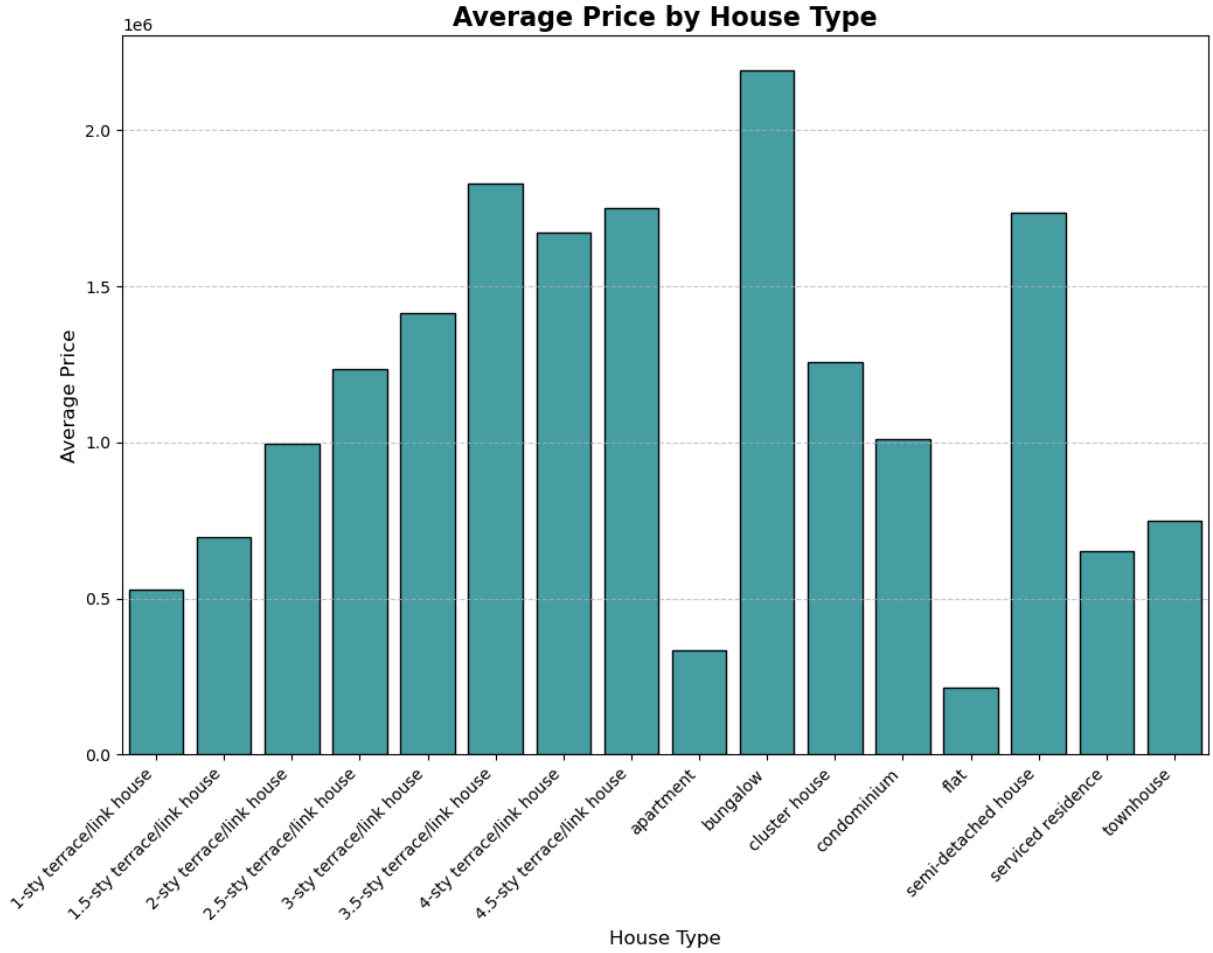

Average Price by Bedrooms
1e6
2.5
2.0
1.5
1.0
0.5
0.0
Average Price
1.0
2.0
3.0
4.0
5.0
6.0
7.0
8.0
9.0
10.0
12.0
13.0
14.0
15.0
16.0
19.0
Bedrooms

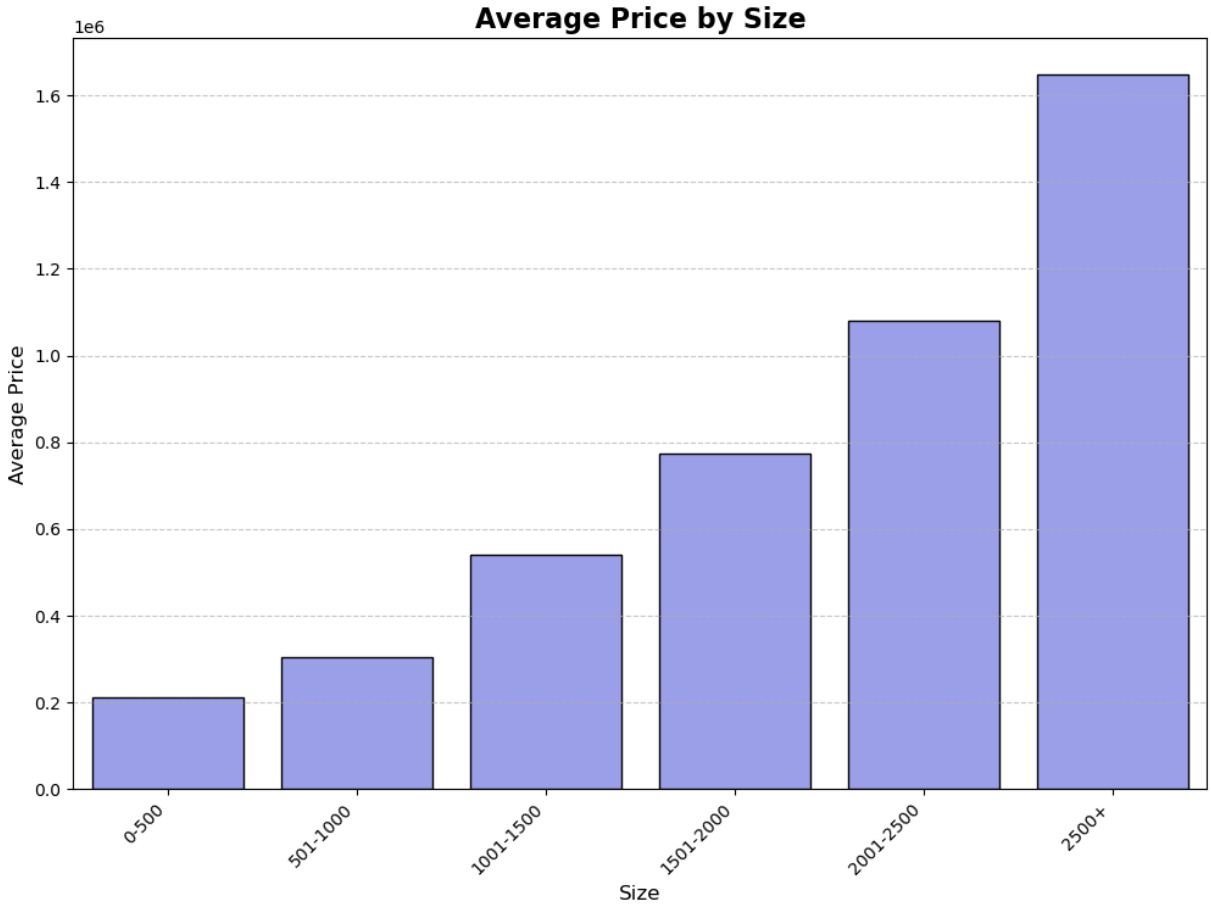

Average Price by Tenure
1e6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Average Price
freehold
leasehold
malay reserved land
private lease scheme
Tenure

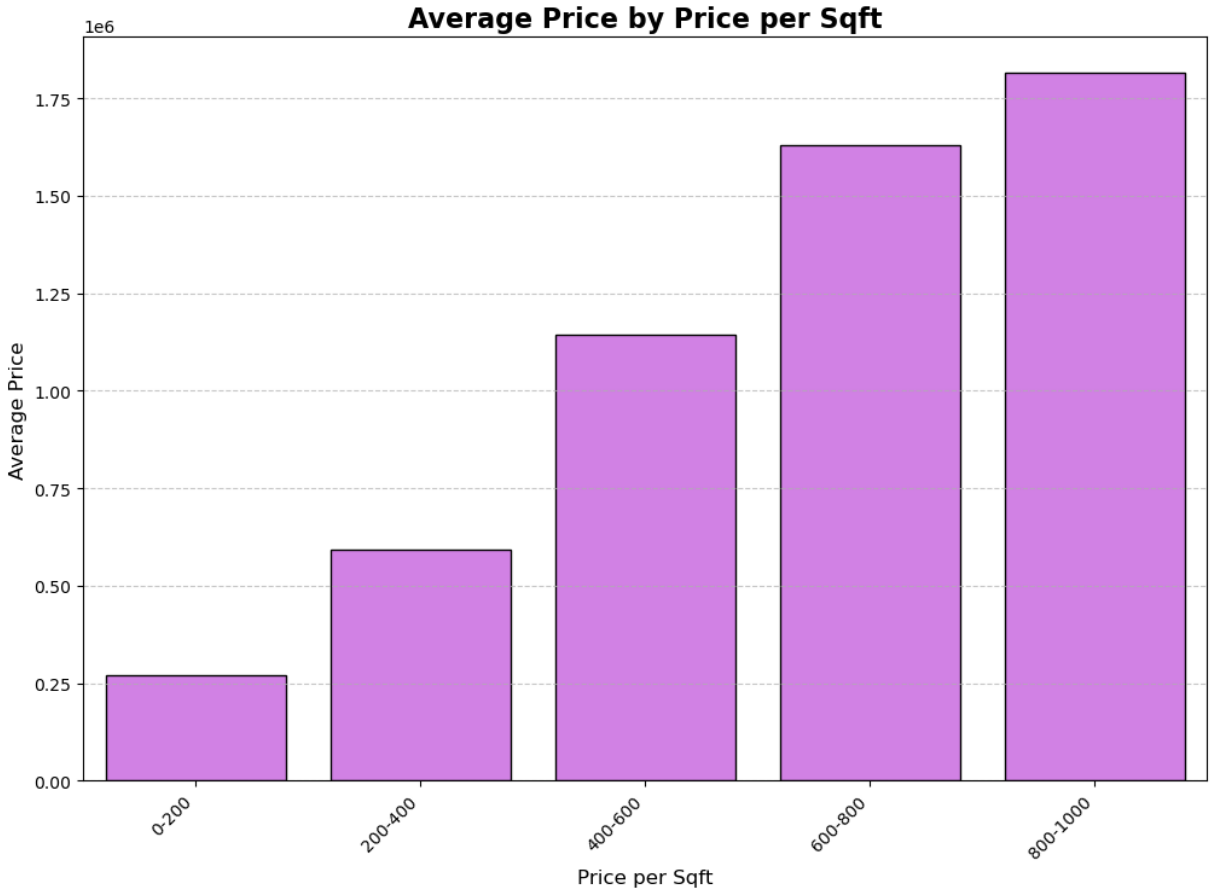

Average Price by Furnished Type
1e6
1.0
0.8
0.6
0.4
0.2
0.0
Average Price
fully furnished
partly furnished
unfurnished
Furnished Type

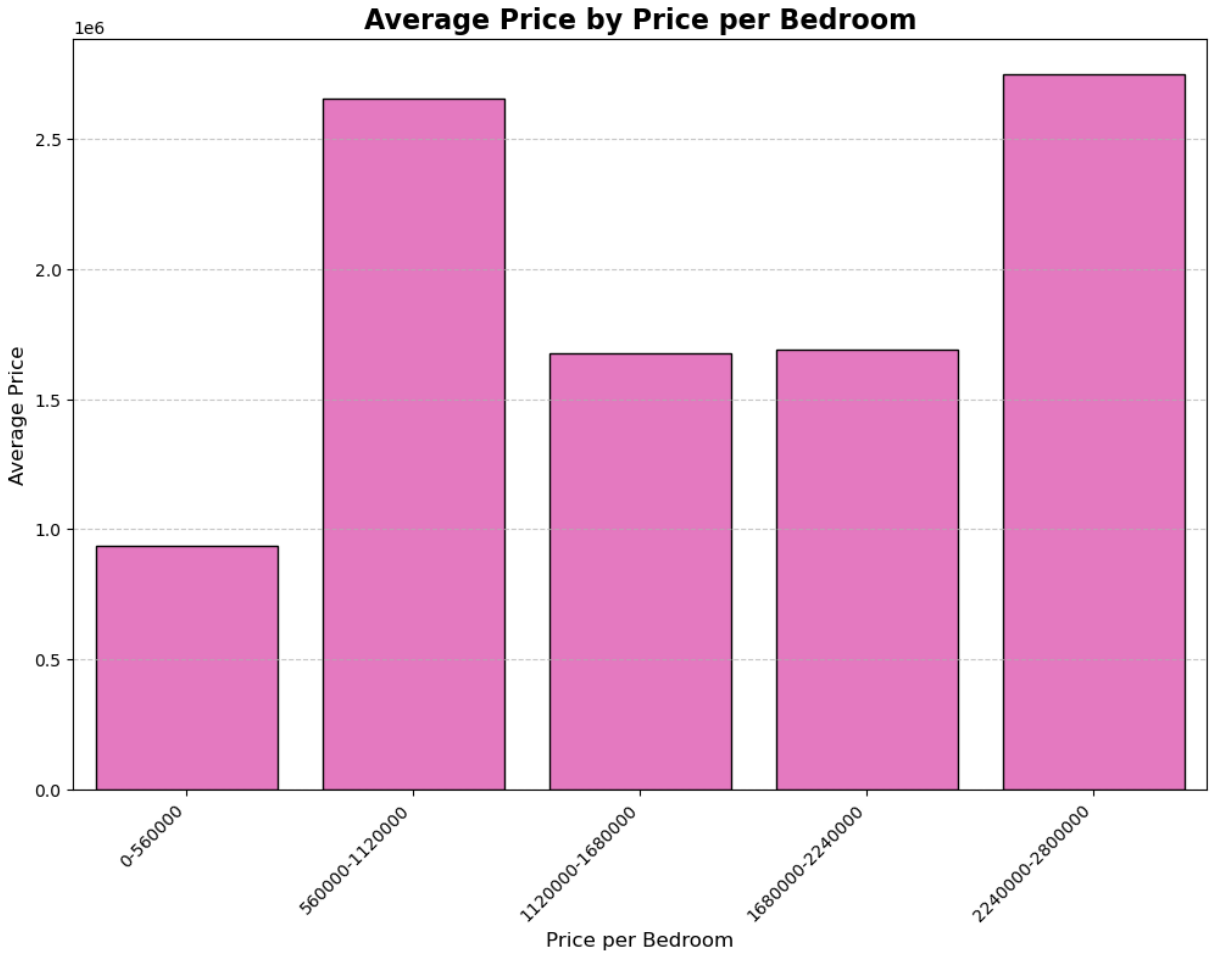

Average Price by Bathrooms
1e6
2.0
1.5
1.0
0.5
0.0
Average Price
1.0
2.0
3.0
4.0
5.0
6.0
7.0
8.0
9.0
13.0
15.0
Bathrooms


Average Price by Car Slots
1e6
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Average Price
1.0
2.0
3.0
4.0
5.0
6.0
7.0
8.0
10.0
12.0
Car Slots

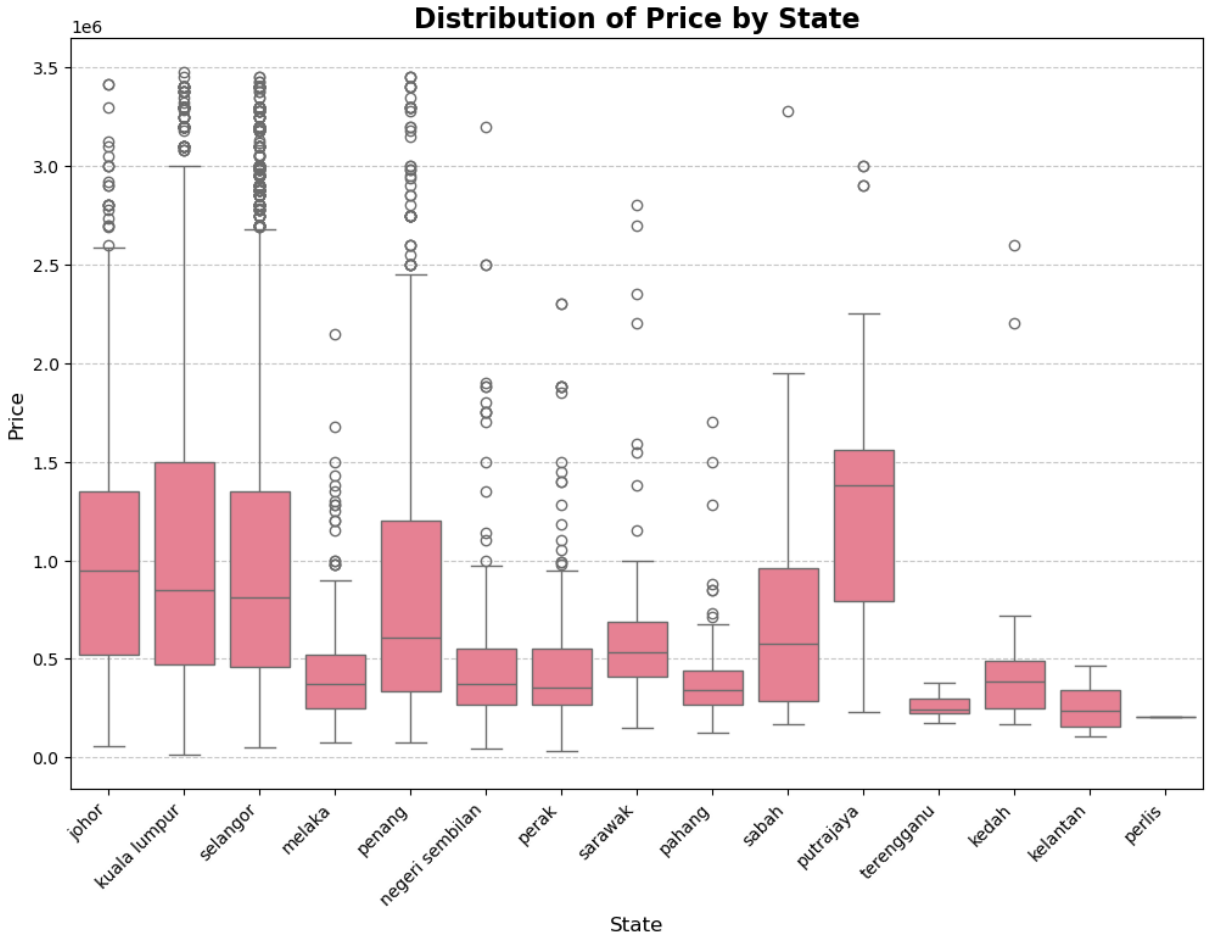

Average Price by House Type
1e6
2.0
1.5
1.0
0.5
0.0
Average Price
1-sty terrace/link house
1.5-sty terrace/link house
2-sty terrace/link house
2.5-sty terrace/link house
3-sty terrace/link house
3.5-sty terrace/link house
4-sty terrace/link house
4.5-sty terrace/link house
apartment
bungalow
cluster house
condominium
flat
semi-detached house
serviced residence
townhouse
House Type

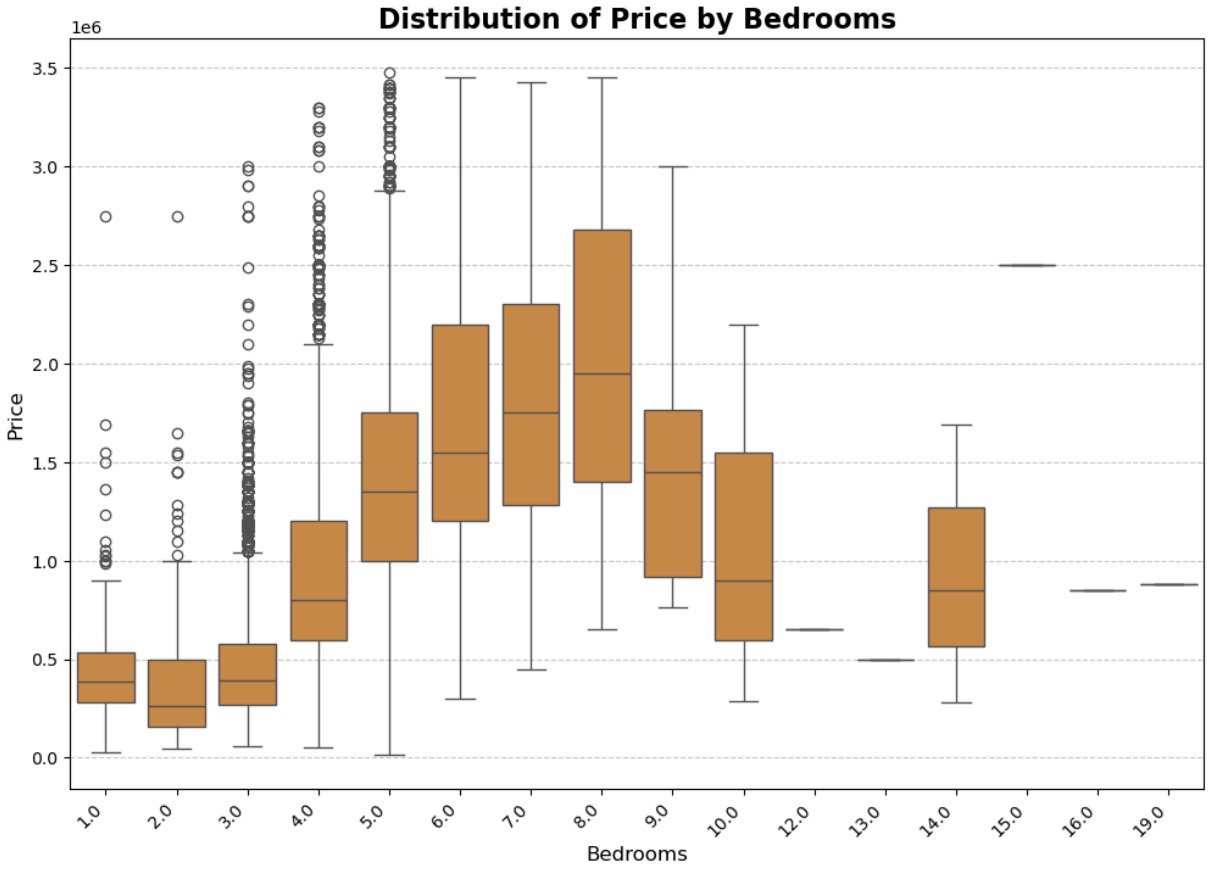

Average Price by Size
1e6
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Average Price
0-500
501-1000
1001-1500
1501-2000
2001-2500
2500+
Size

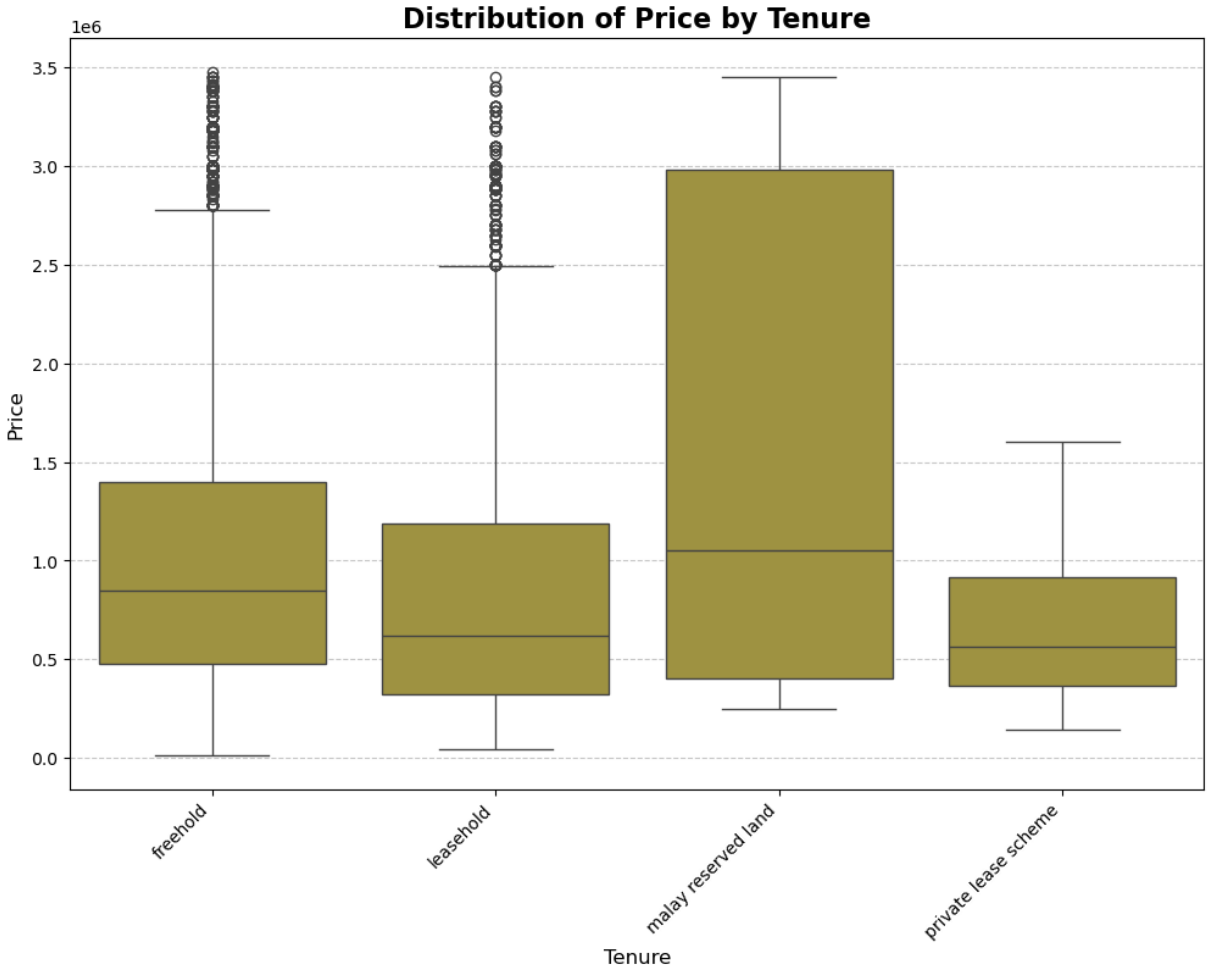

Average Price by Price per Sqft
1e6
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Average Price
0-200
200-400
400-600
600-800
800-1000
Price per Sqft

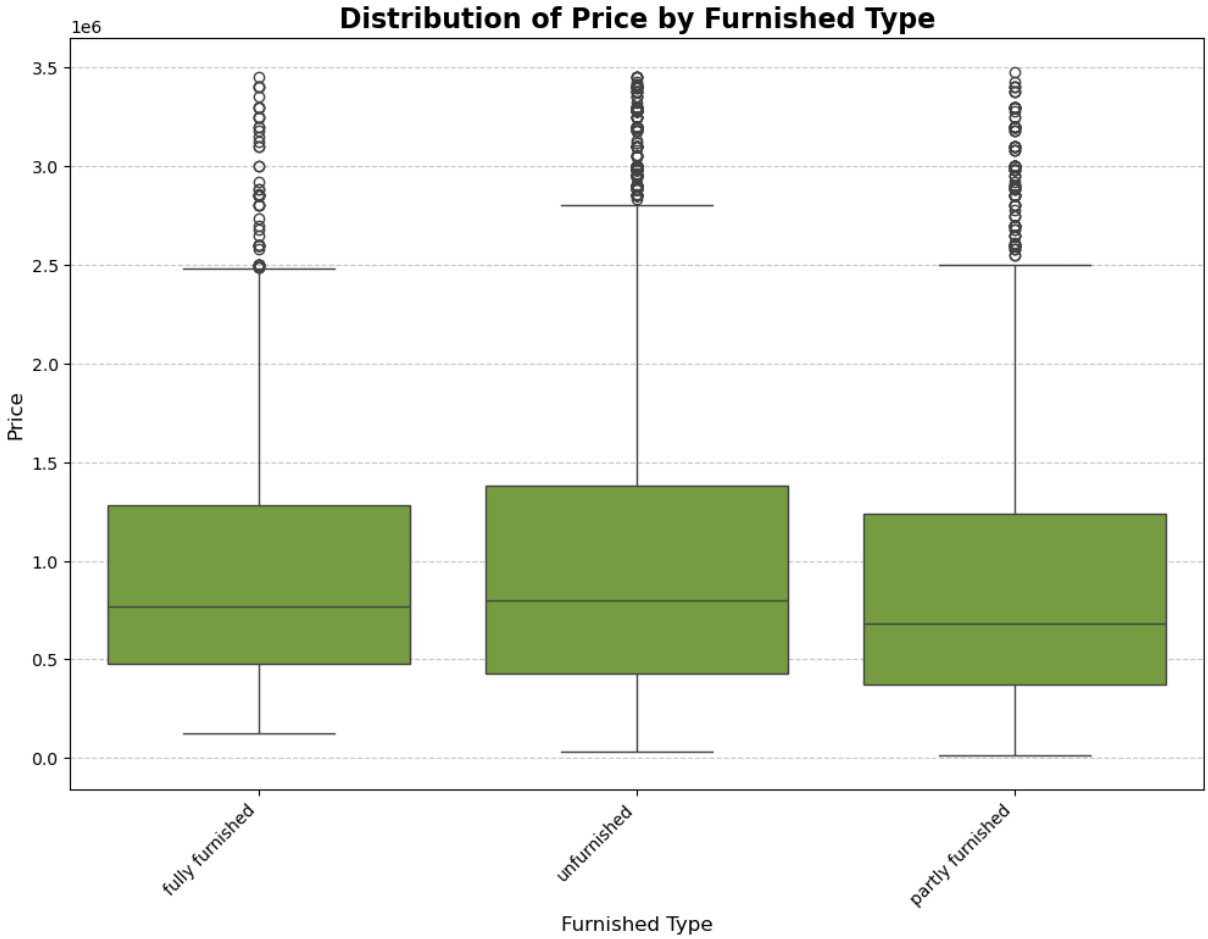


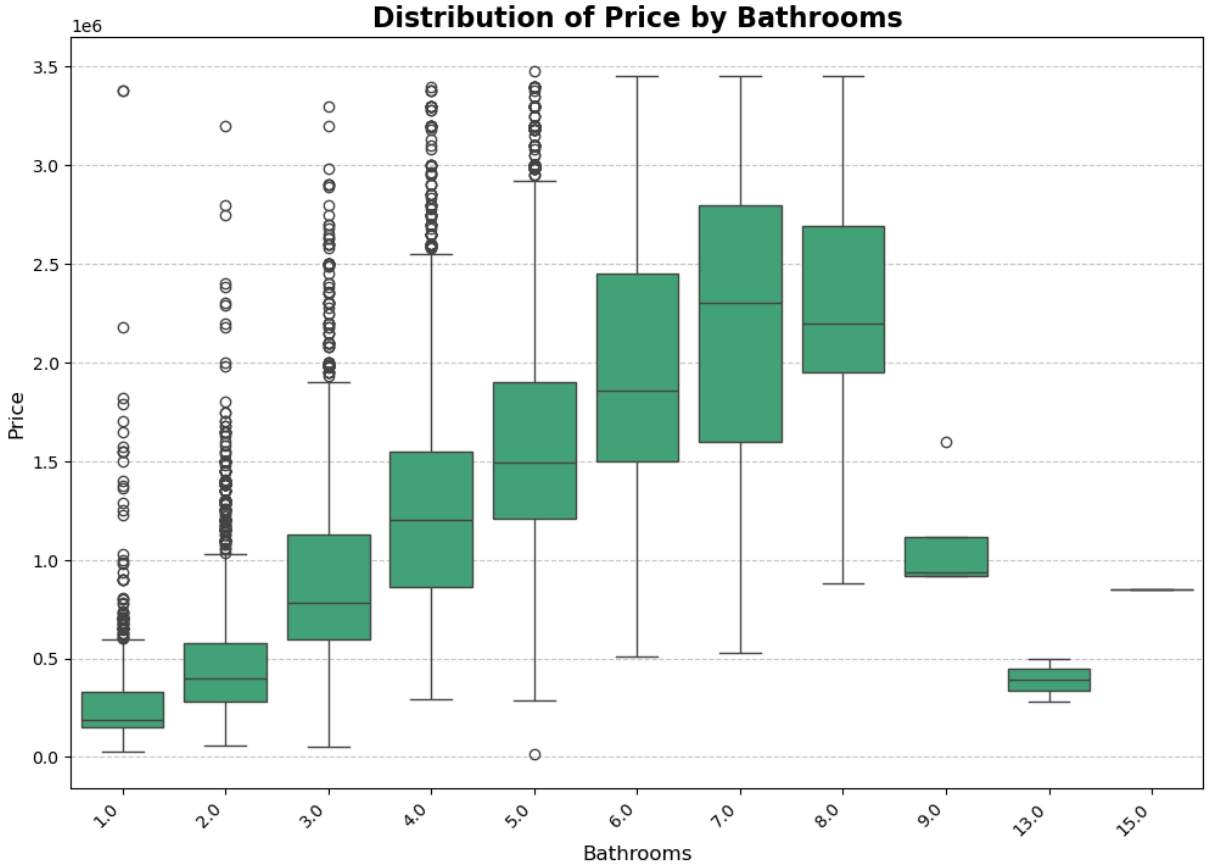
In [25]:
```
# Box plotting
colors = sns.color_palette("husl", n_colors=len(columns_to_plot)+3)

for idx, column in enumerate(columns_to_plot):
    plt.figure(figsize=(12, 8))
    sns.boxplot(data=df, x=column, y='Price', color=colors[idx])
    plt.title(f'Distribution of Price by {column}', fontsize=16, fontweight='bold')
    plt.xlabel(column, fontsize=12)
    plt.ylabel('Price', fontsize=12)
    plt.xticks(rotation=45, ha='right')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()

for idx, binned_column in enumerate(binned_columns):
    plt.figure(figsize=(12, 8))
    sns.boxplot(data=df, x=binned_column, y='Price', color=binned_colors[idx])
    plt.title(f'Distribution of Price by {binned_labels[idx]}', fontsize=16, fontwe
    plt.xlabel(binned_labels[idx], fontsize=12)
    plt.ylabel('Price', fontsize=12)
    plt.xticks(rotation=45, ha='right')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()
```


Distribution of Price by State
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Price
johor
kuala lumpur
selangor
melaka
penang
negeri sembilan
perak
sarawak
pahang
sabah
putrajaya
terengganu
kedah
kelantan
perlis
State

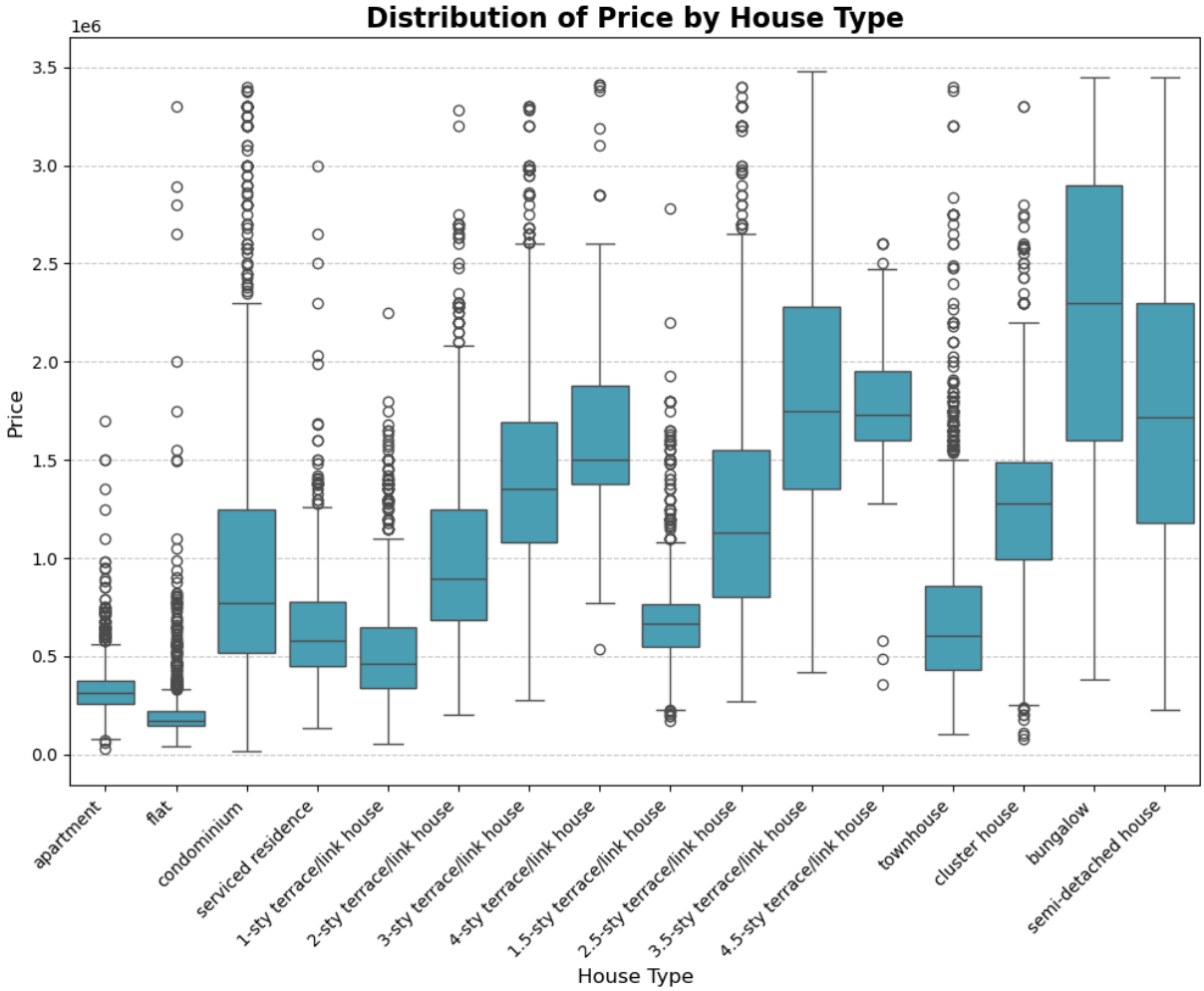

Distribution of Price by Bedrooms
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Price
1.0
2.0
3.0
4.0
5.0
6.0
7.0
8.0
9.0
10.0
12.0
13.0
14.0
15.0
16.0
19.0
Bedrooms

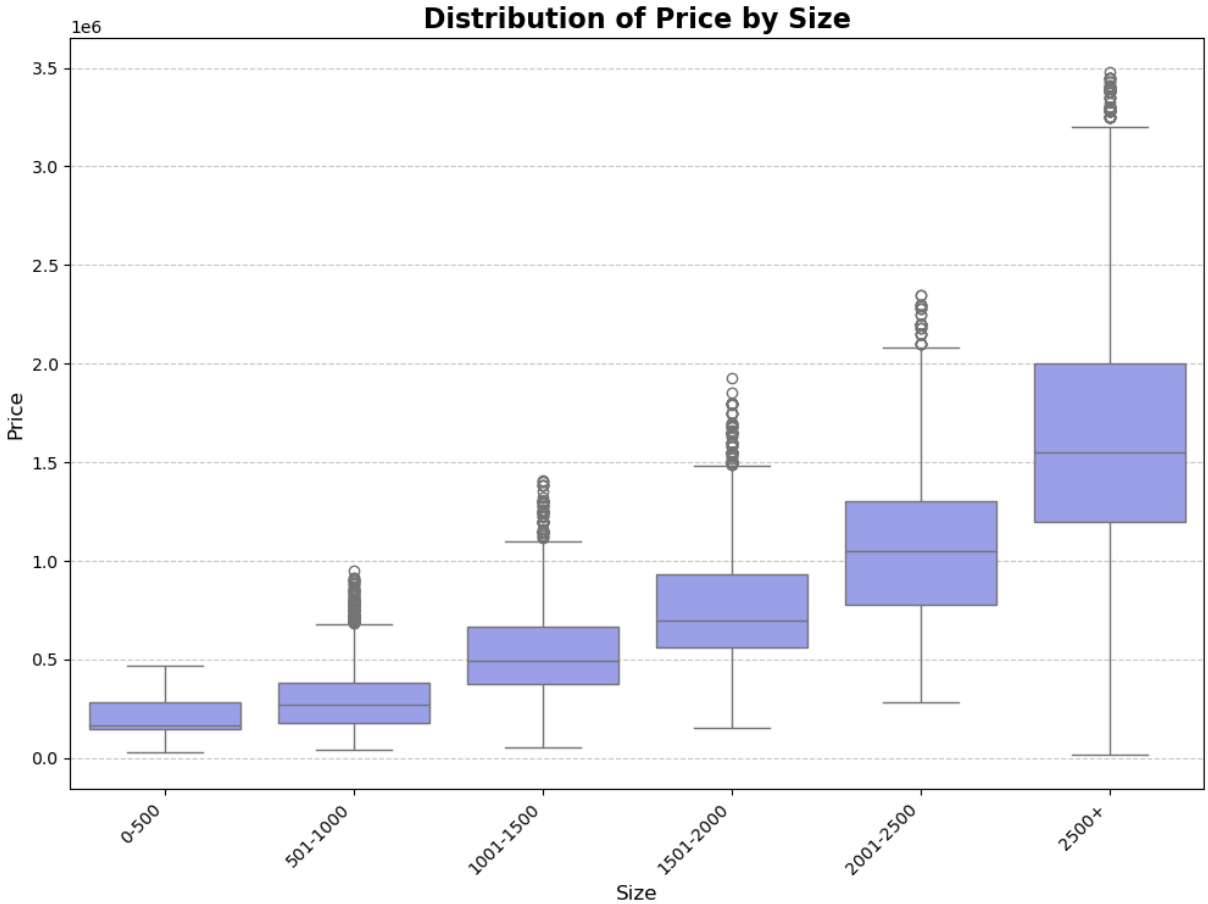

Distribution of Price by Tenure
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Price
freehold
leasehold
malay reserved land
private lease scheme
Tenure

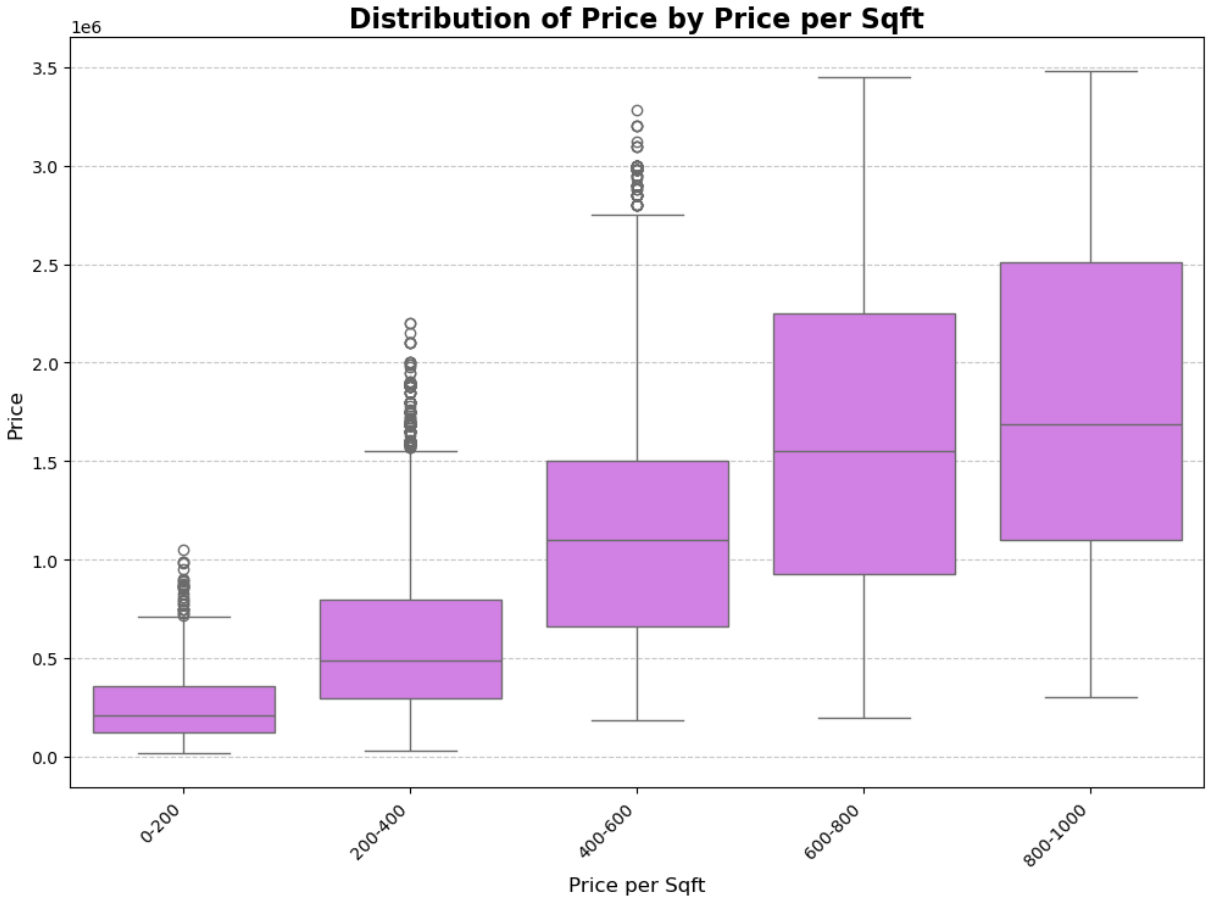

Distribution of Price by Furnished Type
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Price
fully furnished
unfurnished
partly furnished
Furnished Type

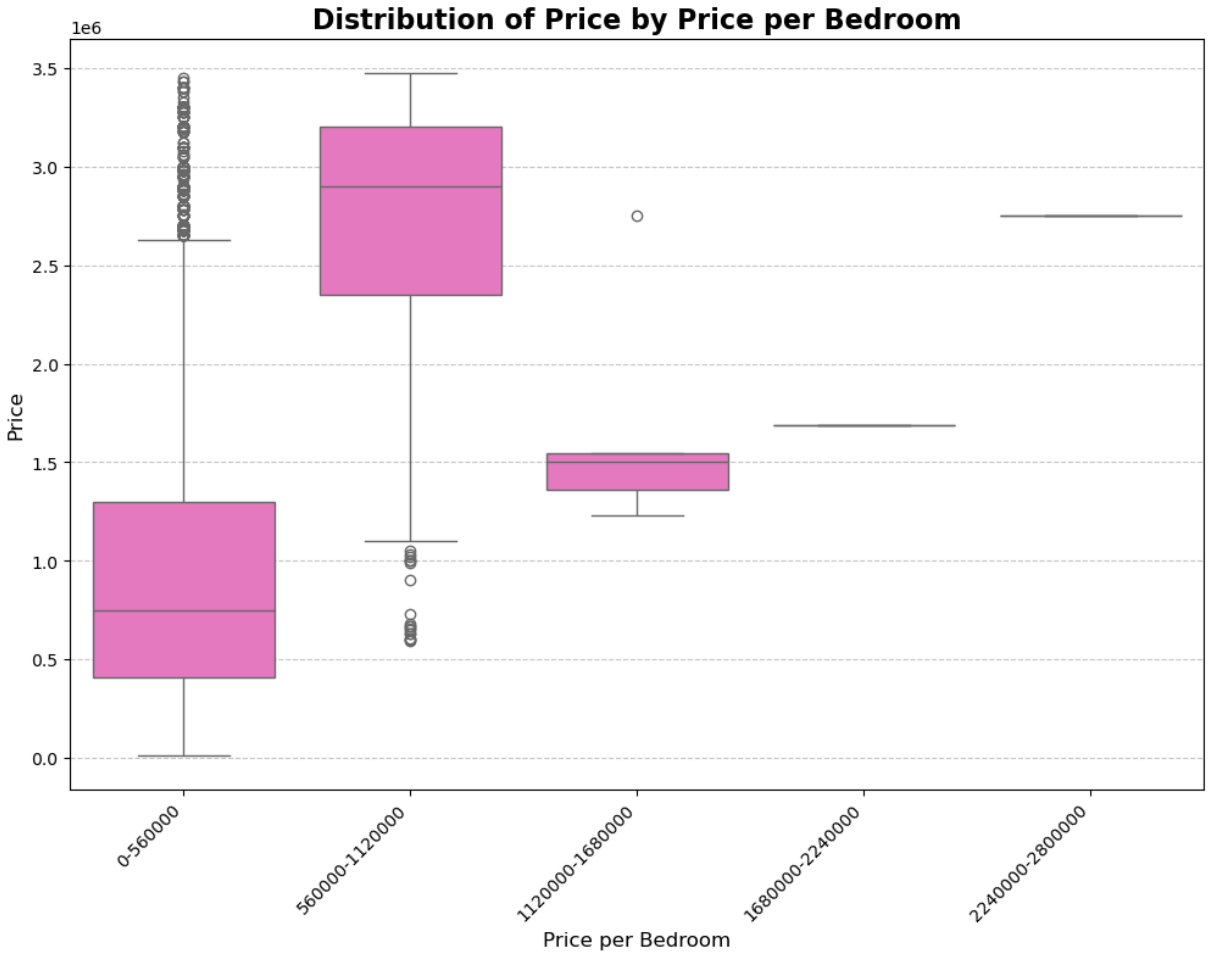

Distribution of Price by Bathrooms
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Price
1.0
2.0
3.0
4.0
5.0
6.0
7.0
8.0
9.0
13.0
15.0
Bathrooms

Distribution of Price by Car Slots
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Price
1.0
2.0
3.0
4.0
5.0
6.0
7.0
8.0
10.0
12.0
Car Slots


Distribution of Price by House Type
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Price
apartment
flat
condominium
serviced residence
1-sty terrace/link house
2-sty terrace/link house
3-sty terrace/link house
4-sty terrace/link house
1.5-sty terrace/link house
2.5-sty terrace/link house
3.5-sty terrace/link house
4.5-sty terrace/link house
townhouse
cluster house
bungalow
semi-detached house
House Type

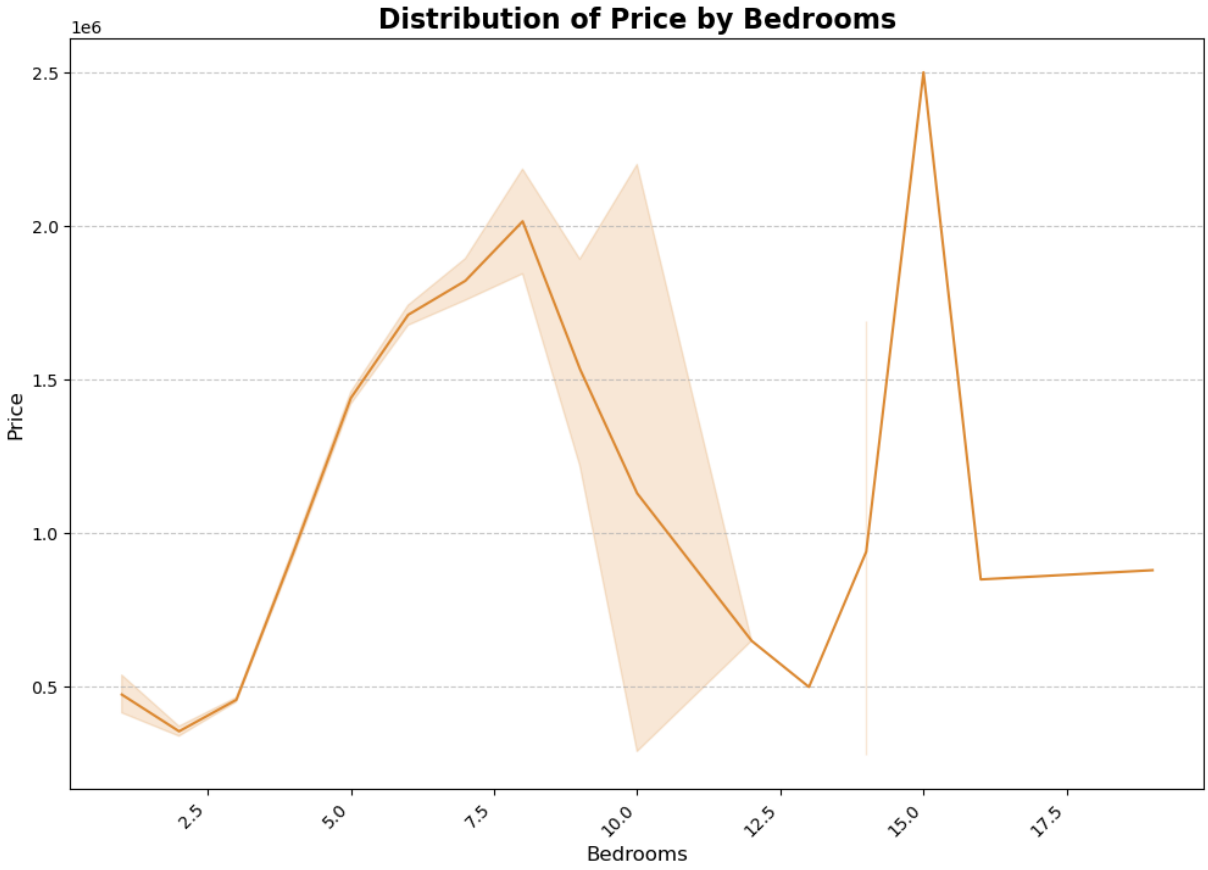

Distribution of Price by Size
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Price
0-500
501-1000
1001-1500
1501-2000
2001-2500
2500+
Size


Distribution of Price by Price per Sqft
1e6
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Price
0-200
200-400
400-600
600-800
800-1000
Price per Sqft

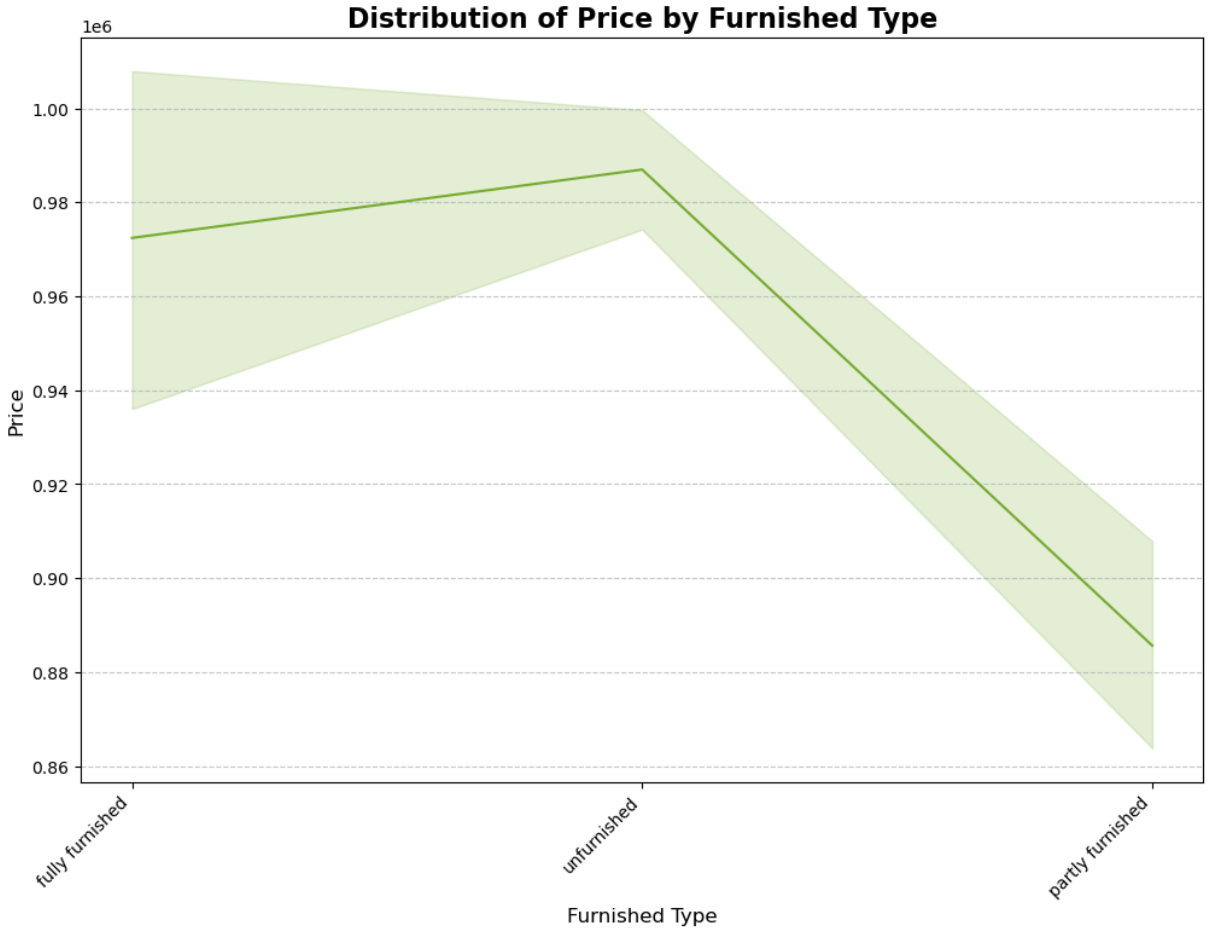


In [26]:
```
# Line plotting
colors = sns.color_palette("husl", n_colors=len(columns_to_plot)+3)

for idx, column in enumerate(columns_to_plot):
    plt.figure(figsize=(12, 8))
    sns.lineplot(data=df, x=column, y='Price', color=colors[idx])
    plt.title(f'Distribution of Price by {column}', fontsize=16, fontweight='bold')
    plt.xlabel(column, fontsize=12)
    plt.ylabel('Price', fontsize=12)
    plt.xticks(rotation=45, ha='right')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()

for idx, binned_column in enumerate(binned_columns):
    plt.figure(figsize=(12, 8))
    sns.lineplot(data=df, x=binned_column, y='Price', color=binned_colors[idx])
    plt.title(f'Avg Price by {binned_labels[idx]}', fontsize=16, fontweight='bold')
    plt.xlabel(binned_labels[idx], fontsize=12)
    plt.ylabel('Price', fontsize=12)
    plt.xticks(rotation=45, ha='right')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.show()
```

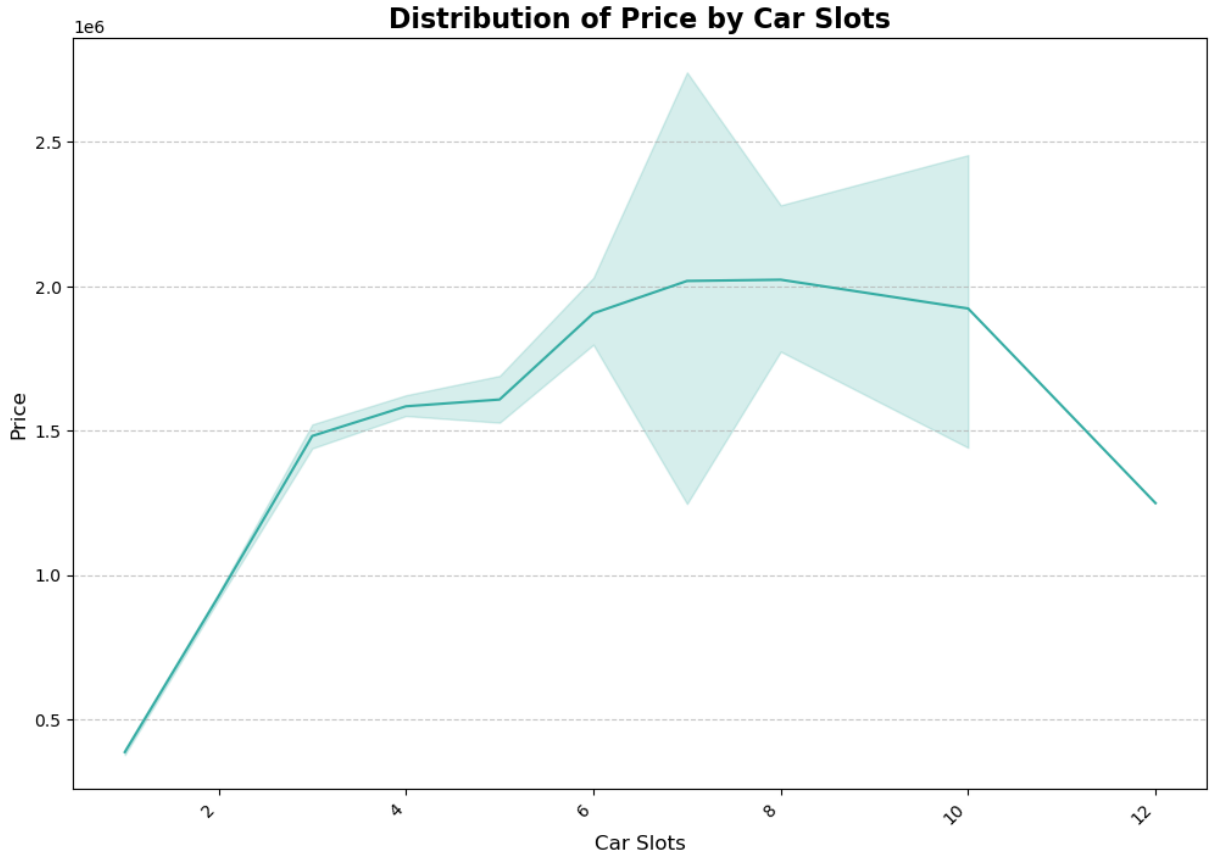
Distribution of Price by State
1e6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
Price
johor
kuala lumpur
selangor
melaka
penang
negeri sembilan
perak
sarawak
pahang
sabah
putrajaya
terengganu
kedah
kelantan
perlis
State

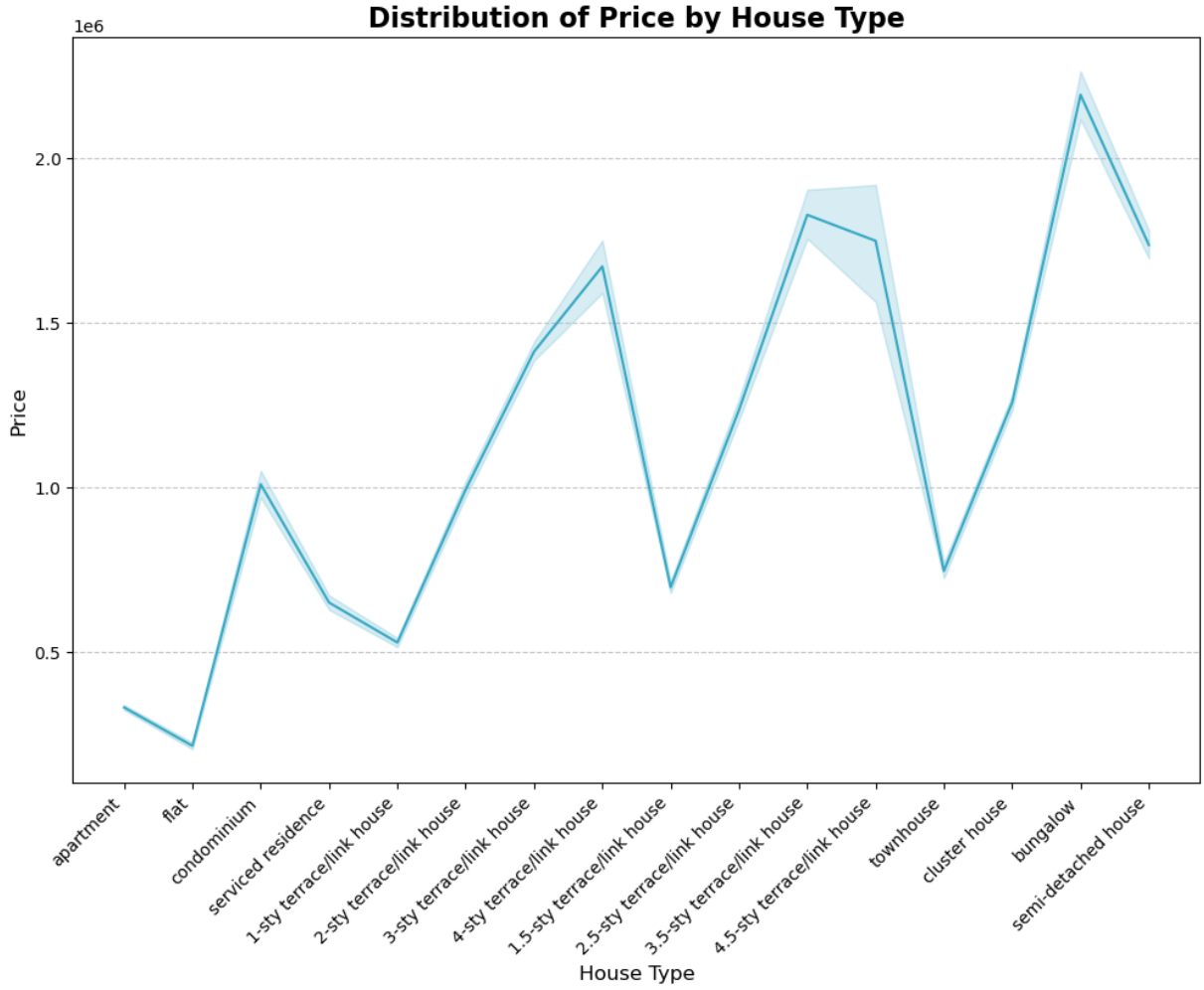

Distribution of Price by Bedrooms
1e6
2.5
2.0
1.5
1.0
0.5
Price
2.5
5.0
7.5
10.0
12.5
15.0
17.5
Bedrooms

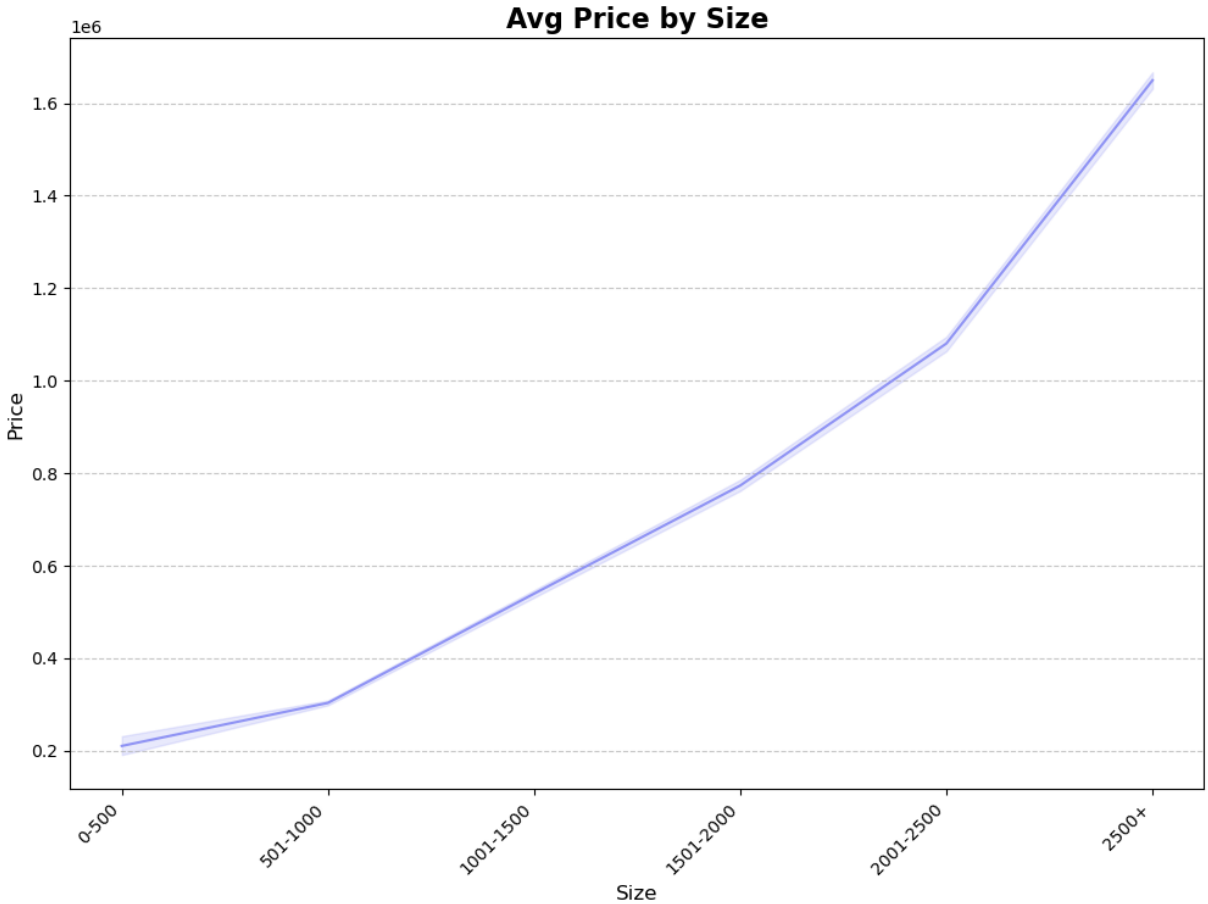
Distribution of Price by Tenure
1e6
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
Price
freehold
leasehold
malay reserved land
private lease scheme
Tenure

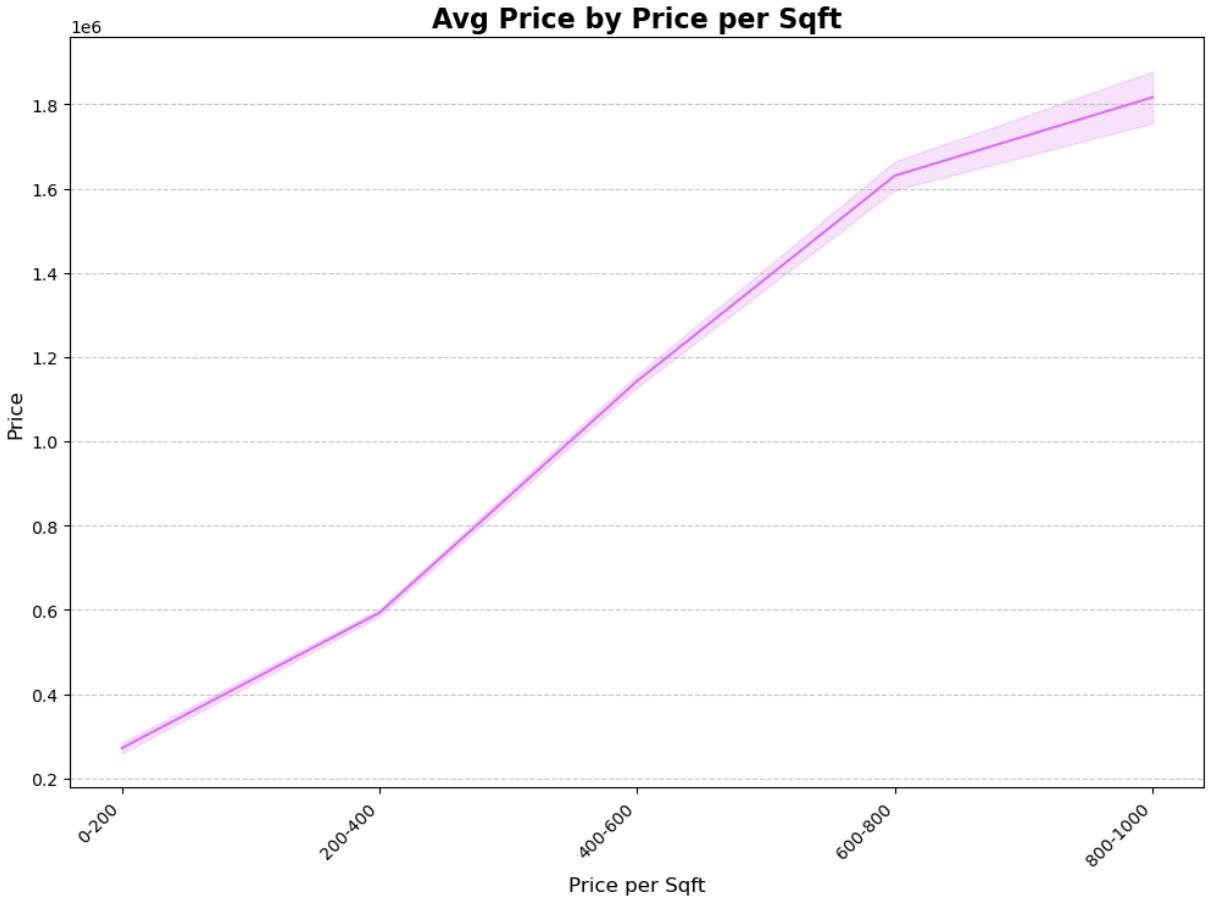

Distribution of Price by Furnished Type
1e6
1.00
0.98
0.96
0.94
0.92
0.90
0.88
0.86
Price
fully furnished
unfurnished
partly furnished
Furnished Type

Distribution of Price by Bathrooms
1e6
2.5
2.0
1.5
1.0
0.5
Price
2
4
6
8
10
12
14
Bathrooms


Distribution of Price by Car Slots
1e6
2.5
2.0
1.5
1.0
0.5
Price
2
4
6
8
10
12
Car Slots


Distribution of Price by House Type
1e6
2.0
1.5
1.0
0.5
Price
apartment
flat
condominium
serviced residence
1-sty terrace/link house
2-sty terrace/link house
3-sty terrace/link house
4-sty terrace/link house
1.5-sty terrace/link house
2.5-sty terrace/link house
3.5-sty terrace/link house
4.5-sty terrace/link house
townhouse
cluster house
bungalow
semi-detached house
House Type


Avg Price by Size
1e6
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
Price
0-500
501-1000
1001-1500
1501-2000
2001-2500
2500+
Size


Avg Price by Price per Sqft
1e6
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0.2
Price
0-200
200-400
400-600
600-800
800-1000
Price per Sqft

🏡 Capstone Project Documentation

1. Data Collection and Preprocessing

📥 Data Sources and Collection Methods

- Data collected from **iProperty Malaysia** using web scraping.
- House types scraped include:
  - 🏢 Apartments, Flats, Condominiums
  - 🏘️ Terrace/Link Houses (1-sty, 2-sty, etc.)
  - 🏠 Townhouses, Cluster Houses, Semi-Detached Houses, Bungalows, etc.
- Attributes scraped:
  - `Price`, `Size`, `Bedrooms`, `Bathrooms`, `Location`, `Facilities`, and `House Type`.

🧹 Data Cleaning Procedures

1. 🔄 **Remove Duplicates:** Avoided by maintaining a set of existing records.
2. 📉 **Handle Missing Values:**

- Removed rows missing critical values (`Price` , `Size`).
- Imputed numerical columns (`Bedrooms` , `Bathrooms` , `Car Slots`) using the median.

3. 🚫 **Address Outliers:** Used the IQR method to clip outliers in `Price` , `Size` , and `Price per Sqft` .
4. 🖊️ **Fix Inconsistent Formatting:** Cleaned categorical values using `.str.strip()` and `.str.lower()` .

🛠️ Feature Engineering

1. 🆕 **Derived Features:**
   - `Price per Bedroom` : Total price divided by the number of bedrooms.
2. 📊 **Binned Features:**
   - **`Size_Bins` :** `0-500` , `501-1000` , ..., `2500+` .
   - **`Price_per_Sqft_Bins` :** `0-200` , `201-400` , ..., `1K+` .
   - **`Price_per_Bedroom_Bins` :** `0-50K` , `51-100K` , ..., `250K+` .

✅ Data Quality Validation

- **Summary Statistics:** Used `df.describe()` for quality checks.
- **Missing Values:** Addressed and validated.

📄 Documentation of Preprocessing Steps

- Code implemented the above cleaning and preprocessing steps systematically.

---

2. Exploratory Data Analysis

📊 Summary Statistics

- Generated metrics for numerical columns:
  - **Mean:** `Price` : RM 1.6M.
  - **Median:** `Bedrooms` : 4.
  - **Range:** `Price` : RM 92K to RM 4.4M.

📈 Distribution Analysis

- Created **Histograms** and **KDE plots**:
  - `Price` is right-skewed; most properties are under RM 2M.
  - `Size` clusters between 1,000–3,000 sq. ft.

🔗 Correlation Analysis

- **Heatmap** findings:
  - `Price` and `Size` are strongly correlated (+0.8).
  - Moderate correlation between `Price per Sqft` and `Bedrooms`.

🔍 Key Trends

- **Scatterplots:**
  - Larger properties generally have higher prices.
  - More bedrooms correlate with higher prices.

📑 Segmentation

- **Size Bins:** Most properties are `1001-1500` sq. ft.
- **Price per Bedroom Bins:** Majority fall in the `51K-100K` range.

📊 Visualizations

1. 📊 **Bar Plots:** Average price by `State`, `Tenure`, and more.
2. 📈 **Scatterplots:** Price trends against numerical variables.
3. 📦 **Box Plots:** Distribution of prices by `Size_Bins` and `Price_per_Sqft_Bins`.
4. 📊 **Histograms:** Distributions of `Price`, `Size`, and other variables.
5. 🌡️ **Heatmaps:** Correlation matrix.

📝 Key Findings

- **Larger properties = Higher prices.**
- **Freehold properties dominate listings.**
- **Price per square foot** varies significantly by state and property type.

🔮 Potential Areas for Deeper Investigation

1. 📍 **Price Variations by Location:** Compare districts within states.
2. 🏫 **Impact of Facilities:** Explore how facilities affect pricing.
3. 📊 **Market Segmentation:** Examine trends in high-demand categories like condominiums or terrace houses.

📝 Notes

- 🎯 **Objective:** The primary goal of this project is to analyze property pricing trends in Malaysia and identify key factors that influence property value.
- 🌍 **Data Scope:**
  - The dataset was scraped from **iProperty Malaysia**.
  - It includes various house types such as 🏢 apartments, 🏠 terrace houses, and 🏡 bungalows.
- ⚠️ **Limitations:**
  - The dataset may not fully represent all property listings in Malaysia.
  - Missing values for certain columns were imputed, which could introduce bias.
  - The analysis assumes the scraped data is accurate and up-to-date.
- 🛠️ **Tools Used:**
  - Python libraries such as:
    - 🕸️ **Selenium** for web scraping.
    - 📊 **Pandas** for data cleaning and manipulation.
    - 📈 **Seaborn & Matplotlib** for visualization.
- 🔮 **Future Steps:**
  - Collect more recent data to expand the analysis over time.
  - Include 🏘️ rental properties in the dataset for rental trend analysis.
  - Apply 🤖 **machine learning models** to predict property prices based on key features.

Legend

- 📥 Data collection
- 🧹 Cleaning
- 📊 Visuals
- 🔍 Trends
- 🔗 Correlation
- 📝 Findings
- 🔮 Deeper investigation

In [27]:
```
df['Facilities_list'] = df['Facilities'].apply(lambda x: eval(x) if isinstance(x, s
mlb = MultiLabelBinarizer()
facilities_encoded = mlb.fit_transform(df['Facilities_list'])
facilities_df = pd.DataFrame(facilities_encoded, columns=mlb.classes_)
df = pd.concat([df.reset_index(drop=True), facilities_df.reset_index(drop=True)], a
```

In [29]:
```
numerical_columns = df.select_dtypes(include=['number']).columns
print("Number of numerical columns:", len(numerical_columns))
```

```
X = df[numerical_columns].drop(['Price', 'Price per Bedroom', 'Room Density'], axis
y = df['Price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

import keras_tuner as kt
from tensorflow.keras import layers, models

def build_model(hp):
    model = models.Sequential()
    model.add(layers.Input(shape=(X_train.shape[1],)))
    for i in range(hp.Int("num_layers", min_value=2, max_value=6)):
        model.add(
            layers.Dense(
                units=hp.Int(f"units_{i}", min_value=128, max_value=4096, step=128)
                activation=hp.Choice("activation", ["relu", "tanh", "sigmoid"])
            )
        )
        if hp.Boolean("batch_norm"):
            model.add(layers.BatchNormalization())
        if hp.Boolean("dropout"):
            model.add(layers.Dropout(rate=hp.Float("dropout_rate", min_value=0.1, m

    model.add(layers.Dense(1))

    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=hp.Float("learning_rate", min_value=1e-4, max_value=1e-2,
        ),
        loss="mse",
        metrics=["mae"]
    )
    return model

early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_mae',
    patience=10,
    restore_best_weights=True
)

tuner = kt.Hyperband(
    build_model,
    objective="val_mae",
    max_epochs=50,
    factor=3,
    directory="my_dir",
    project_name="property_price_prediction"
)

tuner.search(
    X_train,
    y_train,
```

```
    epochs=50,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)

best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]

print(f"Best Hyperparameters:")
for param, value in best_hps.values.items():
    print(f"{param}: {value}")

best_model = tuner.hypermodel.build(best_hps)

history = best_model.fit(
    X_train,
    y_train,
    epochs=100,
    validation_split=0.2,
    batch_size=128,
    verbose=1,
    callbacks=[early_stop]
)

loss, mae = best_model.evaluate(X_test, y_test)
print(f"Test Loss (MSE): {loss:.2f}")
print(f"Test MAE: {mae:.2f}")


loss, mae = best_model.evaluate(X_test, y_test)
print(f"Test Loss (MSE): {loss:.2f}")
print(f"Test MAE: {mae:.2f}")

y_pred = best_model.predict(X_test, batch_size=128)


mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Test Metrics:")
print(f"- Mean Absolute Error (MAE): {mae:.2f}")
print(f"- Mean Squared Error (MSE): {mse:.2f}")
print(f"- Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"- R² Score (Accuracy): {r2:.2f}")

import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss per Epoch')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
```

```
plt.show()
```

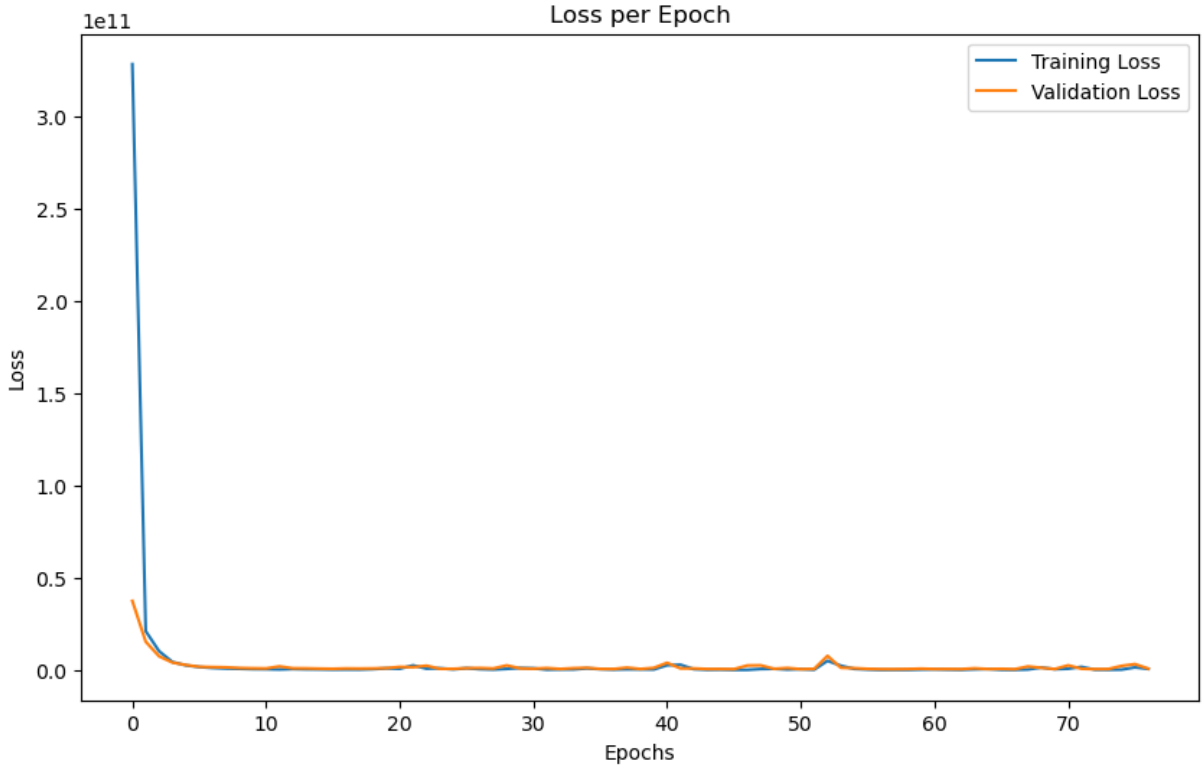
```
Number of numerical columns: 39
Reloading Tuner from my_dir\property_price_prediction\tuner0.json
Best Hyperparameters:
num_layers: 6
units_0: 1920
activation: relu
batch_norm: False
dropout: False
units_1: 256
learning_rate: 0.0007550056505506986
dropout_rate: 0.2
units_2: 2048
units_3: 3200
units_4: 3584
units_5: 3968
tuner/epochs: 50
tuner/initial_epoch: 17
tuner/bracket: 1
tuner/round: 1
tuner/trial_id: 0079
Epoch 1/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 13s 141ms/step - loss: 737297301504.0000 - mae: 595003.43
75 - val_loss: 37533483008.0000 - val_mae: 122858.4531
Epoch 2/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 141ms/step - loss: 24915859456.0000 - mae: 106590.992
2 - val_loss: 15436061696.0000 - val_mae: 83559.7188
Epoch 3/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 11890353152.0000 - mae: 73982.3672
- val_loss: 7507414528.0000 - val_mae: 60741.9766
Epoch 4/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 147ms/step - loss: 5110164480.0000 - mae: 49153.6328
- val_loss: 4129696512.0000 - val_mae: 42683.4062
Epoch 5/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 2910705920.0000 - mae: 38148.9492
- val_loss: 2782742528.0000 - val_mae: 34774.8906
Epoch 6/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 147ms/step - loss: 1866391552.0000 - mae: 30529.4512
- val_loss: 1823575936.0000 - val_mae: 28112.4883
Epoch 7/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 151ms/step - loss: 1220953088.0000 - mae: 24833.2129
- val_loss: 1649386496.0000 - val_mae: 25212.1250
Epoch 8/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 952261952.0000 - mae: 21497.2500 -
val_loss: 1515381248.0000 - val_mae: 26893.2285
Epoch 9/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 735403968.0000 - mae: 19695.3066 -
val_loss: 1144064000.0000 - val_mae: 20935.4062
Epoch 10/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 13s 153ms/step - loss: 596518720.0000 - mae: 16936.0703 -
val_loss: 1005918848.0000 - val_mae: 20135.2871
Epoch 11/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 152ms/step - loss: 538694784.0000 - mae: 16299.7949 -
val_loss: 932651264.0000 - val_mae: 17859.0938
Epoch 12/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 151ms/step - loss: 361021920.0000 - mae: 13288.4033 -
val_loss: 2029123584.0000 - val_mae: 32787.0078
```

```
Epoch 13/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 151ms/step - loss: 712871744.0000 - mae: 19394.4922 -
val_loss: 1016085952.0000 - val_mae: 19281.7891
Epoch 14/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 463691840.0000 - mae: 15501.1650 -
val_loss: 958459392.0000 - val_mae: 17723.5762
Epoch 15/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 151ms/step - loss: 425483648.0000 - mae: 14806.1748 -
val_loss: 816518848.0000 - val_mae: 16977.1895
Epoch 16/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 147ms/step - loss: 302218624.0000 - mae: 12276.7266 -
val_loss: 725696064.0000 - val_mae: 14902.7588
Epoch 17/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 324792576.0000 - mae: 12830.2305 -
val_loss: 914364928.0000 - val_mae: 18125.7168
Epoch 18/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 299015040.0000 - mae: 12370.3604 -
val_loss: 853806144.0000 - val_mae: 16821.8574
Epoch 19/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 21s 152ms/step - loss: 558418560.0000 - mae: 16979.8594 -
val_loss: 930170112.0000 - val_mae: 20339.2715
Epoch 20/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 147ms/step - loss: 871078400.0000 - mae: 21382.7871 -
val_loss: 1081269120.0000 - val_mae: 19948.9902
Epoch 21/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 150ms/step - loss: 687809152.0000 - mae: 18817.0781 -
val_loss: 1755764096.0000 - val_mae: 28545.8613
Epoch 22/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 151ms/step - loss: 2835280384.0000 - mae: 37986.6484
- val_loss: 1606565888.0000 - val_mae: 29002.4805
Epoch 23/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 13s 152ms/step - loss: 1042148544.0000 - mae: 22815.5527
- val_loss: 2404827648.0000 - val_mae: 32342.2129
Epoch 24/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 13s 155ms/step - loss: 1331257600.0000 - mae: 26740.1875
- val_loss: 731072512.0000 - val_mae: 16009.6650
Epoch 25/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 474497920.0000 - mae: 15507.2812 -
val_loss: 709258112.0000 - val_mae: 16441.2637
Epoch 26/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 904002624.0000 - mae: 21375.7832 -
val_loss: 1047352000.0000 - val_mae: 20954.1855
Epoch 27/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 20s 146ms/step - loss: 510821216.0000 - mae: 16370.0518 -
val_loss: 1136259072.0000 - val_mae: 23532.2734
Epoch 28/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 152ms/step - loss: 350456672.0000 - mae: 13326.6816 -
val_loss: 907283008.0000 - val_mae: 20790.9395
Epoch 29/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 444055904.0000 - mae: 14412.0898 -
val_loss: 2566496000.0000 - val_mae: 36440.3984
Epoch 30/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 147ms/step - loss: 1369044608.0000 - mae: 26806.4980
- val_loss: 964420032.0000 - val_mae: 18538.3613
Epoch 31/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 146ms/step - loss: 1253163392.0000 - mae: 24948.7715
```

```
- val_loss: 769444544.0000 - val_mae: 16671.3301
Epoch 32/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 21s 146ms/step - loss: 271465696.0000 - mae: 11645.1719 -
val_loss: 1124480896.0000 - val_mae: 22876.4180
Epoch 33/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 21s 146ms/step - loss: 471065824.0000 - mae: 15307.1094 -
val_loss: 651439104.0000 - val_mae: 14867.3350
Epoch 34/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 21s 147ms/step - loss: 260918672.0000 - mae: 11439.1582 -
val_loss: 1049215936.0000 - val_mae: 22123.7090
Epoch 35/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 146ms/step - loss: 829302464.0000 - mae: 20706.8809 -
val_loss: 1238303616.0000 - val_mae: 25566.3594
Epoch 36/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 812002112.0000 - mae: 19401.5898 -
val_loss: 657175424.0000 - val_mae: 14904.0957
Epoch 37/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 152ms/step - loss: 357056896.0000 - mae: 13171.7441 -
val_loss: 614364800.0000 - val_mae: 13417.7354
Epoch 38/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 261610192.0000 - mae: 11026.9150 -
val_loss: 1380777984.0000 - val_mae: 25563.6621
Epoch 39/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 20s 147ms/step - loss: 533676864.0000 - mae: 16625.1680 -
val_loss: 627642240.0000 - val_mae: 15001.3203
Epoch 40/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 314636640.0000 - mae: 12551.4424 -
val_loss: 1208164480.0000 - val_mae: 25368.6172
Epoch 41/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 1643748096.0000 - mae: 27173.8203
- val_loss: 3978269440.0000 - val_mae: 47041.5938
Epoch 42/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 150ms/step - loss: 3991372032.0000 - mae: 44819.5508
- val_loss: 940572352.0000 - val_mae: 18477.1562
Epoch 43/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 570444224.0000 - mae: 16733.5918 -
val_loss: 918946304.0000 - val_mae: 18284.8281
Epoch 44/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 251977312.0000 - mae: 11160.8213 -
val_loss: 608241088.0000 - val_mae: 14317.3066
Epoch 45/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 21s 149ms/step - loss: 220517504.0000 - mae: 10399.3486 -
val_loss: 605264704.0000 - val_mae: 13395.3633
Epoch 46/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 21s 148ms/step - loss: 169223616.0000 - mae: 9158.1133 -
val_loss: 543701952.0000 - val_mae: 12550.5537
Epoch 47/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 123347184.0000 - mae: 7728.1387 -
val_loss: 2515617280.0000 - val_mae: 37916.6133
Epoch 48/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 20s 146ms/step - loss: 722543936.0000 - mae: 18953.0273 -
val_loss: 2594948608.0000 - val_mae: 39514.3086
Epoch 49/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 147ms/step - loss: 1159798528.0000 - mae: 24250.1094
- val_loss: 719479616.0000 - val_mae: 17420.3730
Epoch 50/100
```

```
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 150ms/step - loss: 282045344.0000 - mae: 11927.9443 -
val_loss: 1170352256.0000 - val_mae: 23717.4277
Epoch 51/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 150ms/step - loss: 679875136.0000 - mae: 18248.3418 -
val_loss: 533646336.0000 - val_mae: 12900.3467
Epoch 52/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 211002032.0000 - mae: 10206.9209 -
val_loss: 677784320.0000 - val_mae: 17141.9121
Epoch 53/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 150ms/step - loss: 2873541888.0000 - mae: 31813.8926
- val_loss: 7735800320.0000 - val_mae: 66849.5469
Epoch 54/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 151ms/step - loss: 4021602816.0000 - mae: 43729.9180
- val_loss: 1289942016.0000 - val_mae: 24178.8398
Epoch 55/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 591979328.0000 - mae: 17071.2832 -
val_loss: 1063972608.0000 - val_mae: 19101.9492
Epoch 56/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 358290688.0000 - mae: 12949.0869 -
val_loss: 686050560.0000 - val_mae: 15540.1152
Epoch 57/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 162153184.0000 - mae: 8945.1621 -
val_loss: 536096224.0000 - val_mae: 12643.0273
Epoch 58/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 180477600.0000 - mae: 9482.6094 -
val_loss: 509161376.0000 - val_mae: 11675.4072
Epoch 59/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 145ms/step - loss: 114320184.0000 - mae: 7312.6875 -
val_loss: 598753664.0000 - val_mae: 13577.2842
Epoch 60/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 21s 149ms/step - loss: 288996992.0000 - mae: 12185.6357 -
val_loss: 792727232.0000 - val_mae: 17501.7305
Epoch 61/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 516045760.0000 - mae: 16186.2549 -
val_loss: 600229696.0000 - val_mae: 13728.9355
Epoch 62/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 152ms/step - loss: 241497248.0000 - mae: 11008.5420 -
val_loss: 594664768.0000 - val_mae: 13556.6611
Epoch 63/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 151ms/step - loss: 225001488.0000 - mae: 10657.9678 -
val_loss: 552922112.0000 - val_mae: 14097.4385
Epoch 64/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 148ms/step - loss: 197374112.0000 - mae: 9650.4590 -
val_loss: 989842624.0000 - val_mae: 21473.5039
Epoch 65/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 150ms/step - loss: 909452352.0000 - mae: 20922.5117 -
val_loss: 600670272.0000 - val_mae: 13933.1738
Epoch 66/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 151ms/step - loss: 183442112.0000 - mae: 9528.5576 -
val_loss: 692363392.0000 - val_mae: 14999.2617
Epoch 67/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 13s 152ms/step - loss: 163387696.0000 - mae: 9050.0898 -
val_loss: 507443072.0000 - val_mae: 12492.4014
Epoch 68/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 13s 152ms/step - loss: 212128368.0000 - mae: 9815.3135 -
val_loss: 2002201984.0000 - val_mae: 33751.5000
```

```
Epoch 69/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 1332859136.0000 - mae: 26673.0000
- val_loss: 1254996992.0000 - val_mae: 21350.6152
Epoch 70/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 147ms/step - loss: 679959040.0000 - mae: 18373.4824 -
val_loss: 601500480.0000 - val_mae: 14221.2881
Epoch 71/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 151ms/step - loss: 400678624.0000 - mae: 14055.9482 -
val_loss: 2593970944.0000 - val_mae: 36618.1289
Epoch 72/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 3064812288.0000 - mae: 37628.3945
- val_loss: 755973888.0000 - val_mae: 15653.0742
Epoch 73/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 147ms/step - loss: 330719072.0000 - mae: 12622.6963 -
val_loss: 544095296.0000 - val_mae: 12581.0469
Epoch 74/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 151ms/step - loss: 185192624.0000 - mae: 9649.1436 -
val_loss: 587678336.0000 - val_mae: 13139.0020
Epoch 75/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 149ms/step - loss: 196223536.0000 - mae: 9703.8682 -
val_loss: 2305323776.0000 - val_mae: 38038.8633
Epoch 76/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 144ms/step - loss: 1365702272.0000 - mae: 26932.0625
- val_loss: 3263785728.0000 - val_mae: 38209.8047
Epoch 77/100
82/82 ━━━━━━━━━━━━━━━━━━━━ 12s 144ms/step - loss: 1169418368.0000 - mae: 23974.6113
- val_loss: 798535360.0000 - val_mae: 16104.0459
102/102 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 389913312.0000 - mae: 12044.4189
Test Loss (MSE): 444323936.00
Test MAE: 12529.38
102/102 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - loss: 389913312.0000 - mae: 12044.4189
Test Loss (MSE): 444323936.00
Test MAE: 12529.38
26/26 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step
Test Metrics:
- Mean Absolute Error (MAE): 12529.38
- Mean Squared Error (MSE): 444323894.66
- Root Mean Squared Error (RMSE): 21078.99
- R² Score (Accuracy): 1.00
```



In [31]:
```
best_model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	
dense_7 (Dense)	(None, 1920)	
dense_8 (Dense)	(None, 256)	
dense_9 (Dense)	(None, 2048)	
dense_10 (Dense)	(None, 3200)	
dense_11 (Dense)	(None, 3584)	1
dense_12 (Dense)	(None, 3968)	1
dense_13 (Dense)	(None, 1)	

Total params: 100,042,757 (381.63 MB)

Trainable params: 33,347,585 (127.21 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 66,695,172 (254.42 MB)

In [32]:
```
rf_model = RandomForestRegressor(random_state=24)
rf_model.fit(X_train, y_train)
```

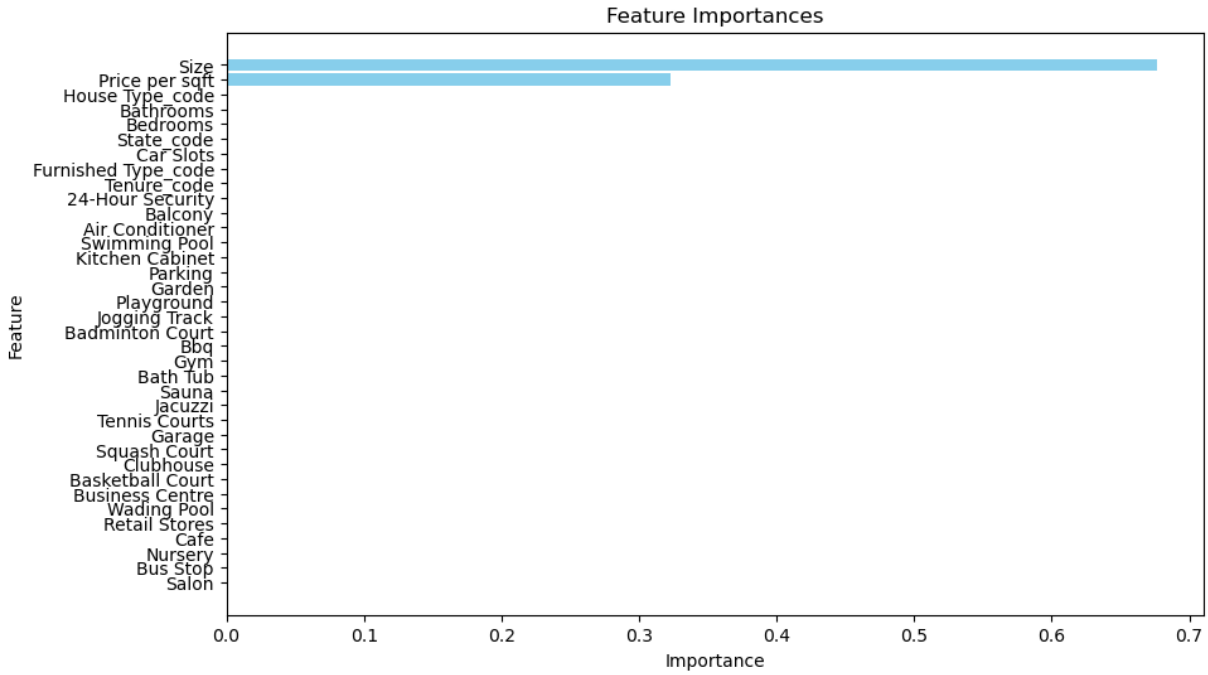
Out[32]:

```
▾ RandomForestRegressor
RandomForestRegressor(random_state=24)
```

In [33]:

```
importances = rf_model.feature_importances_
feature_names = df[numerical_columns].drop(['Price', 'Price per Bedroom', 'Room Den
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': impor

feature_importance_df = feature_importance_df.sort_values(by='Importance', ascendin

print(feature_importance_df)

plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df['Feature'], feature_importance_df['Importance'], col
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.title('Feature Importances')
plt.gca().invert_yaxis()
plt.show()
```

	Feature	Importance
1	Size	0.676374
4	Price per sqft	0.323155
8	House Type_code	0.000063
2	Bathrooms	0.000047
0	Bedrooms	0.000043
5	State_code	0.000037
3	Car Slots	0.000036
7	Furnished Type_code	0.000025
6	Tenure_code	0.000021
9	24-Hour Security	0.000015
12	Balcony	0.000013
10	Air Conditioner	0.000012
33	Swimming Pool	0.000012
25	Kitchen Cabinet	0.000011
27	Parking	0.000011
21	Garden	0.000010
28	Playground	0.000010
24	Jogging Track	0.000010
11	Badminton Court	0.000008
15	Bbq	0.000008
22	Gym	0.000008
14	Bath Tub	0.000007
31	Sauna	0.000007
23	Jacuzzi	0.000007
34	Tennis Courts	0.000007
20	Garage	0.000007
32	Squash Court	0.000006
19	Clubhouse	0.000006
13	Basketball Court	0.000005
17	Business Centre	0.000005
35	Wading Pool	0.000004
29	Retail Stores	0.000004
18	Cafe	0.000004
26	Nursery	0.000002
16	Bus Stop	0.000001
30	Salon	0.000001



In [34]:
```
joblib.dump(best_model, "property_price_model.pkl")
joblib.dump(scaler, "scaler.pkl")
```

Out[34]: ['scaler.pkl']

In [ ]: